Atelier Python pour le développement IA

Guide pas à pas, de l'installation de Python jusqu'à un assistant qui répond en s'appuyant sur des fiches

Ce que vous allez construire

Vous partez d'un dossier presque vide : un fichier de quinze recettes, un fichier de vingt questions d'utilisateurs, et un petit programme, `modele_local.py`, qui joue le rôle d'un fournisseur de modèle de langage. Il répond au même format que l'API d'OpenAI, mais il tourne sur votre machine, sans compte, sans carte bancaire et sans réseau vers l'extérieur.

À la fin, votre dossier contient dix-neuf fichiers que vous aurez écrits vous-mêmes : un script qui vérifie votre installation, un module de recherche dans les recettes, une configuration propre, deux façons d'appeler un modèle, un classeur de questions qui ne croit que les réponses valides, des tests automatiques, une mesure de qualité sur vingt questions, et un assistant en ligne de commande qui répond à partir des fiches.

Le parcours

Partie	Ce que vous faites
0	Installer Python et VS Code, ouvrir un terminal
1	Préparer le dossier de travail et le parcourir en ligne de commande
2	Créer un environnement virtuel, installer les bibliothèques, figer leurs versions
3	Essayer Python en direct dans l'interpréteur
4	Variables, types, listes, dictionnaires, conditions, boucles, fonctions
5	Lire un fichier JSON et en tirer des chiffres
6	Ranger son code dans un module et l'importer
7	Lire une erreur, la rattraper, en lever une
8	Sortir la configuration et la clé du code
9	Lancer le modèle local et l'appeler en HTTP avec `requests`
10	Faire le même appel avec le SDK `openai`, tenir une conversation
11	Décrire la réponse attendue avec Pydantic et rejeter ce qui n'est pas valide
12	Écrire des tests avec pytest, y compris sans appeler le modèle
13	Mesurer la qualité du classement sur des questions dont on connaît la réponse
14	Chercher les bonnes fiches et répondre en s'appuyant dessus
15	Faire le point sur le dossier et rendre le travail

Comment lire ce guide

- Chaque commande est donnée deux fois, dans un tableau à deux colonnes : **Windows (PowerShell)** à gauche, **macOS (Terminal)** à droite. Prenez la colonne de votre machine.
- Les fenêtres noires sont de vraies captures de terminal. L'invite est en gris, ce qu'il faut taper est en jaune, le reste est ce que la machine a répondu. Les captures ont été faites sous Windows : sur Mac, l'invite ressemble à `(.venv) vous@mac atelier-python %`, le reste est identique sauf mention contraire.
- Les fenêtres claires montrent le fichier que vous éditez dans VS Code. Les lignes surlignées en vert sont celles à ajouter à cette étape, les lignes barrées en rouge celles à retirer. Une ligne en gris italique, du type « (lignes 1 à 9 inchangées) », signifie que vous ne touchez pas à cette partie.
- **Tapez le code, ne le copiez pas depuis le PDF.** Le copier-coller depuis un PDF remplace souvent les guillemets droits par des guillemets typographiques et casse l'indentation, et Python refuse les deux.
- Les encadrés « À vous » sont des exercices courts. Notez vos réponses au fur et à mesure dans un fichier `reponses.txt`, placé dans le dossier de travail : il fait partie de ce que vous rendez.

Partie 0. Installer les outils

Trois outils suffisent pour tout l'atelier : Python lui-même, un éditeur de code, et un terminal. Aucun droit administrateur n'est nécessaire si vous suivez les options indiquées.

0.1 Vérifier si Python est déjà là

Ouvrez un terminal (voir 0.4 si vous ne savez pas comment) et tapez :

Windows (PowerShell)	macOS (Terminal)
`python --version`	`python3 --version`

Si la réponse est `Python 3.10` ou un numéro plus grand, passez directement à 0.3. Si la commande est inconnue, ou si Windows ouvre le Microsoft Store, ou si la version est plus ancienne, installez Python comme ci-dessous.

0.2 Installer Python 3.12

Sous Windows

1. Allez sur **python.org**, menu *Downloads*, et téléchargez l'installateur Windows 64 bits de la dernière version 3.12.
2. Lancez-le. Sur le premier écran, **cochez « Add python.exe to PATH »**, tout en bas. C'est la case qui permet de taper `python` dans n'importe quel terminal. Si vous l'oubliez, rien de ce qui suit ne marchera.
3. Ne cochez pas l'option d'installation pour tous les utilisateurs : l'installation se fait alors dans votre profil, sans droit administrateur.
4. Cliquez sur *Install Now*, attendez la fin, fermez.
5. **Fermez tous les terminaux ouverts** et rouvrez-en un : un terminal ne voit le nouveau PATH qu'à son ouverture.

Sous macOS

1. Allez sur **python.org**, menu *Downloads*, et téléchargez l'installateur macOS de la dernière version 3.12 (un fichier `.pkg`, valable pour les Mac Intel comme Apple Silicon).
2. Ouvrez-le et suivez les écrans sans rien changer.
3. À la fin, le Finder ouvre un dossier *Python 3.12* : double-cliquez sur `Install Certificates.command`. Sans cela, les téléchargements de pip peuvent échouer sur une erreur de certificat.
4. Rouvrez le Terminal. Sur Mac, la commande s'appelle `python3` tant que l'environnement virtuel n'est pas activé : `python` seul n'existe pas, ou pointe vers autre chose.

0.3 Installer VS Code et son extension Python

1. Téléchargez Visual Studio Code sur **code.visualstudio.com** et installez-le. Sous Windows, prenez la version « User Installer », qui ne demande pas de droit administrateur, et laissez cochée l'option qui ajoute *Ouvrir avec Code* au menu contextuel.
2. Dans VS Code, ouvrez le panneau des extensions (l'icône des quatre carrés dans la barre de gauche), cherchez **Python** et installez celle publiée par Microsoft. Elle apporte la coloration, la détection des erreurs pendant la frappe, et le choix de l'interpréteur.

0.4 Ouvrir un terminal

Windows (PowerShell)	macOS (Terminal)
Touche Windows, tapez **PowerShell**, Entrée. Ou, dans VS Code : menu *Terminal*, *Nouveau terminal*. Vérifiez que l'invite commence par `PS` : si elle commence par `C:\>` sans PS, c'est l'ancienne invite de commandes, et plusieurs commandes du guide n'y marchent pas.	Cmd+Espace, tapez **Terminal**, Entrée. Ou, dans VS Code : menu *Terminal*, *Nouveau terminal*. L'invite se termine par `%` : c'est le shell zsh, celui de macOS par défaut.

Le terminal de VS Code est le plus pratique : il s'ouvre directement dans le dossier ouvert dans l'éditeur, et vous voyez le code et son résultat dans la même fenêtre.

Partie 1. Préparer le dossier de travail

Un projet Python est un dossier. Tout ce que vous écrivez, tout ce que vous installez pour ce projet, et toutes les commandes que vous lancez partent de ce dossier. La moitié des erreurs de débutant viennent d'un terminal ouvert au mauvais endroit : cette partie vous apprend à toujours savoir où vous êtes.

1.1 Récupérer les fichiers de départ

1. Téléchargez l'archive `atelier-depart.zip` fournie par votre formateur.
2. Décompressez-la dans un dossier à vous, par exemple `Documents`. Sous Windows : clic droit, *Extraire tout*. Sous macOS : double-clic sur l'archive.
3. Renommez le dossier obtenu en **`atelier-python`**. Pas d'espace ni d'accent dans le nom : certains outils les gèrent mal.
4. Dans VS Code : menu *Fichier*, *Ouvrir le dossier*, choisissez `atelier-python`. Si VS Code demande si vous faites confiance aux auteurs, répondez oui.
5. Ouvrez un terminal dans VS Code (menu *Terminal*, *Nouveau terminal*). Il s'ouvre dans le dossier du projet.

1.2 Vérifier Python et pip

pip est l'installateur de bibliothèques de Python. On l'appelle toujours à travers Python, avec `python -m pip` : l'option `-m` lance un module de ce Python-là, ce qui garantit que les bibliothèques s'installent pour le bon interpréteur quand il y en a plusieurs sur la machine.

Windows (PowerShell)	macOS (Terminal)
`python --version` `python -m pip --version`	`python3 --version` `python3 -m pip --version`





La deuxième réponse dit aussi **où** se trouve pip : ici, dans le Python installé pour toute la machine. Retenez ce chemin, il va changer à la partie 2.

1.3 Savoir où l'on est, et ce qu'il y a autour

Windows (PowerShell)	macOS (Terminal)

`Get-Location`
`Get-ChildItem -Name`
`Get-ChildItem donnees -Name`
	`pwd`
	`ls`
	`ls donnees`



La première commande affiche le dossier courant, la deuxième son contenu, la troisième le contenu du sous-dossier `donnees`. Vous devez voir `modele_local.py` et `donnees`, puis `questions.csv` et `recettes.json`. Si ce n'est pas le cas, votre terminal n'est pas dans le bon dossier : fermez-le et rouvrez-le depuis VS Code, dossier `atelier-python` ouvert.

1.4 Regarder un fichier de données, et le piège de l'encodage

Un fichier texte est une suite d'octets. L'**encodage** est la règle qui dit quel octet correspond à quelle lettre. Les fichiers de l'atelier sont en **UTF-8**, la norme du web et de Python. Affichez les premières lignes des questions :

Windows (PowerShell)	macOS (Terminal)
`Get-Content donnees\questions.csv -TotalCount 4`	`head -n 4 donnees/questions.csv`

```
Windows PowerShell
PS C:\Users\Public\Documents\M1\atelier-python> Get-Content donnees\questions.csv -TotalCount 4
question;intention_attendue
J'ai des courgettes et des oeufs, je fais quoi ce soir ?;recette_frigo
Il me reste du riz et des champignons, une idÃ©e de dÃ®ner ?;recette_frigo
Je n'ai pas le temps, une recette en moins de 20 minutes ?;recette_frigo
PS C:\Users\Public\Documents\M1\atelier-python>
```

Sous Windows, les accents sortent abîmés : « idÃ©e » au lieu de « idée ». Le fichier n'est pas cassé. C'est Windows PowerShell qui le lit avec l'ancien encodage de Windows. On lui dit explicitement lequel employer :

Windows (PowerShell)	macOS (Terminal)
`Get-Content donnees\questions.csv -Encoding UTF8 -TotalCount 4`	rien à changer : le Terminal de macOS lit déjà l'UTF-8

```
Windows PowerShell
PS C:\Users\Public\Documents\M1\atelier-python> Get-Content donnees\questions.csv -Encoding UTF8 -TotalCount 4
question;intention_attendue
J'ai des courgettes et des oeufs, je fais quoi ce soir ?;recette_frigo
Il me reste du riz et des champignons, une idée de dîner ?;recette_frigo
Je n'ai pas le temps, une recette en moins de 20 minutes ?;recette_frigo
PS C:\Users\Public\Documents\M1\atelier-python>
```

Retenez la leçon, elle reviendra en Python : **quand on lit un fichier texte, on précise toujours l'encodage**. Vous le ferez avec `encoding="utf-8"` à chaque `open()`.

Le fichier est un CSV : une ligne par question, les colonnes séparées par un point-virgule. La première ligne donne le nom des colonnes : la question, et l'intention que l'on attend du classement. Il servira à la partie 13.

1.5 Les séparateurs de chemin

Windows sépare les dossiers par une barre oblique inverse, `donnees\questions.csv`. macOS et Linux emploient la barre oblique, `donnees/questions.csv`. PowerShell accepte les deux, le Terminal de macOS seulement la seconde. Dans votre code Python, vous n'écrirez ni l'une ni l'autre à la main : la partie 5 montre comment `pathlib` s'en occupe.

> **À vous (E1)**
>
> Créez le fichier `reponses.txt` dans `atelier-python` (dans VS Code : clic droit dans l'explorateur de fichiers, *Nouveau fichier*). Notez-y la version de Python affichée chez vous, et le chemin de pip. Ce chemin contient-il le mot `.venv` ?

Partie 2. L'environnement virtuel et les bibliothèques

Un projet d'IA dépend de bibliothèques : `openai` pour parler au modèle, `pydantic` pour valider ses réponses, `pytest` pour tester. Chaque projet en veut des versions précises. Si tout s'installe dans le Python de la machine, deux projets finissent par se marcher dessus. La solution est l'**environnement virtuel** : un dossier, propre au projet, qui contient sa copie de Python et ses bibliothèques.

2.1 Un script qui dit la vérité sur votre installation

Avant de créer l'environnement, écrivez un petit script qui affiche quel Python tourne et quelles bibliothèques il voit. Vous le relancerez plusieurs fois pour constater les changements. Dans VS Code, créez le fichier `verifier.py` et tapez :

verifier.py

```
# verifier.py : dit quel Python tourne et quelles bibliothèques sont installées
import sys
from importlib.metadata import version, PackageNotFoundError

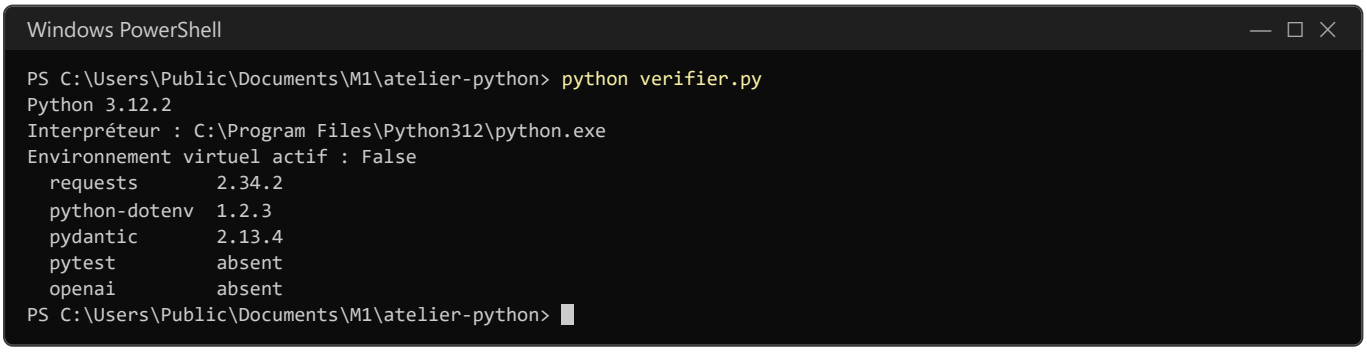
print("Python", sys.version.split()[0])
print("Interpréteur :", sys.executable)
print("Environnement virtuel actif :", sys.prefix != sys.base_prefix)

for nom in ["requests", "python-dotenv", "pydantic", "pytest", "openai"]:
    try:
        print(f"  {nom:<14} {version(nom)}")
    except PackageNotFoundError:
        print(f"  {nom:<14} absent")
```

- `import sys` charge le module qui décrit l'interpréteur en cours. `sys.executable` est le chemin du programme Python qui exécute le script.
- `sys.prefix != sys.base_prefix` vaut `True` uniquement dans un environnement virtuel : les deux chemins diffèrent alors.
- La boucle essaie de lire la version de chaque bibliothèque. Si elle n'est pas installée, `version()` lève une erreur `PackageNotFoundError`, que le bloc `try / except` rattrape pour afficher « absent ». Vous verrez ce mécanisme en détail à la partie 7.
- `{nom:<14}` aligne le nom à gauche sur 14 caractères, pour que les versions tombent en colonne.

Enregistrez (Ctrl+S sous Windows, Cmd+S sous macOS), puis lancez :

Windows (PowerShell)	macOS (Terminal)
`python verifier.py`	`python3 verifier.py`



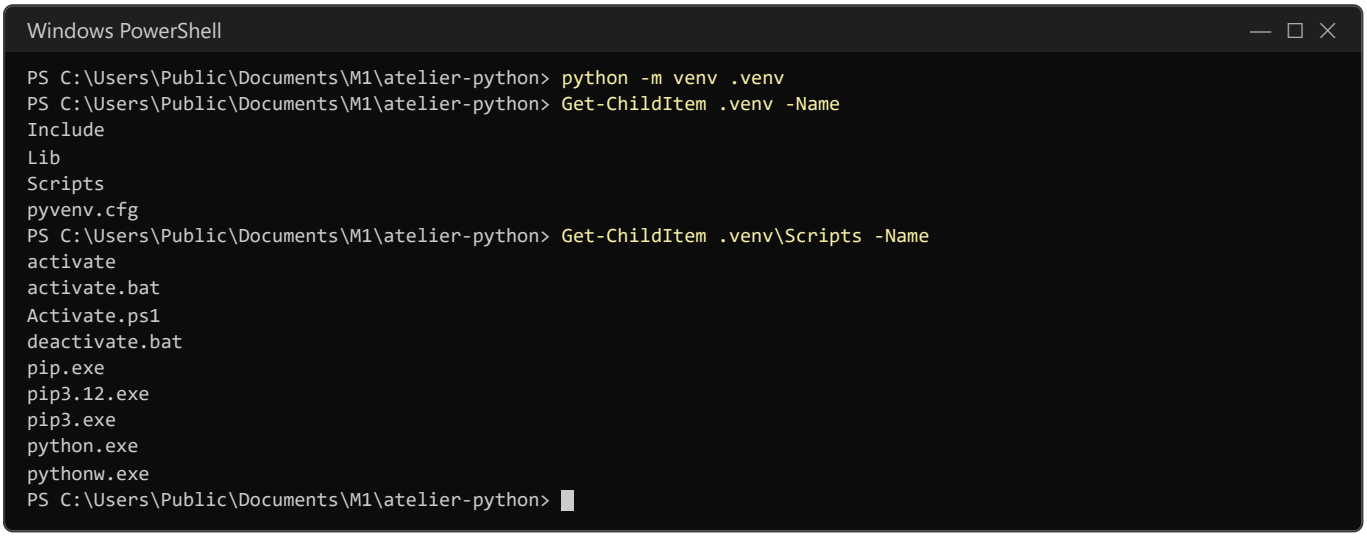
Lecture : le script tourne avec le Python de la machine, pas dans un environnement virtuel (`False`). Selon ce qui a déjà été installé sur votre poste, certaines bibliothèques apparaissent, d'autres sont absentes : c'est justement ce hasard que l'environnement virtuel supprime.

2.2 Créer l'environnement

Le module `venv`, livré avec Python, fabrique l'environnement. Par convention, on l'appelle `.venv` et on le place à la racine du projet. Le point en tête du nom le signale comme un dossier technique.

Windows (PowerShell)	macOS (Terminal)

`python -m venv .venv`
`Get-ChildItem .venv -Name`
`Get-ChildItem .venv\Scripts -Name`
	`python3 -m venv .venv`
	`ls .venv`
	`ls .venv/bin`



La création ne dit rien quand elle réussit. Le dossier contient la bibliothèque propre au projet (`Lib`, vide pour l'instant en dehors de pip) et un dossier de programmes : `Scripts` sous Windows, `bin` sous macOS. On y trouve un `python`, un `pip`, et les scripts d'**activation**.

2.3 Activer l'environnement

Activer, c'est dire au terminal : « à partir de maintenant, quand je tape `python`, prends celui de `.venv` ». L'activation ne vaut que pour ce terminal, et jusqu'à sa fermeture.

Windows (PowerShell)	macOS (Terminal)
`.venv\Scripts\Activate.ps1`	`source .venv/bin/activate`

Sous macOS, l'invite prend aussitôt le préfixe `(.venv)` : passez directement à 2.4. Sous Windows, il est probable que la première tentative soit refusée :

```
Windows PowerShell
PS C:\Users\Public\Documents\M1\atelier-python> .venv\Scripts\Activate.ps1
.venv\Scripts\Activate.ps1 : Impossible de charger le fichier
C:\Users\Public\Documents\M1\atelier-python\.venv\Scripts\Activate.ps1, car l'exécution de scripts est désactivée sur
ce système. Pour plus d'informations, consultez about_Execution_Policies à l'adresse
https://go.microsoft.com/fwlink/?LinkID=135170.
Au caractère Ligne:1 : 1
+ .venv\Scripts\Activate.ps1
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : Erreur de sécurité : (:) [], PSSecurityException
    + FullyQualifiedErrorId : UnauthorizedAccess
PS C:\Users\Public\Documents\M1\atelier-python>
```

Ce n'est pas une panne. Par sécurité, Windows interdit par défaut à PowerShell d'exécuter des scripts, et `Activate.ps1` en est un. On autorise les scripts présents sur votre disque, pour votre compte seulement, ce qui ne demande pas de droit administrateur. Cette commande ne se tape qu'une fois par machine :

Windows (PowerShell)	macOS (Terminal)
`Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned -Force` `Get-ExecutionPolicy -Scope CurrentUser`	sans objet : macOS n'a pas cette règle

```
Windows PowerShell
PS C:\Users\Public\Documents\M1\atelier-python> Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
-Force
PS C:\Users\Public\Documents\M1\atelier-python> Get-ExecutionPolicy -Scope CurrentUser
RemoteSigned
PS C:\Users\Public\Documents\M1\atelier-python>
```

`RemoteSigned` autorise les scripts écrits sur votre machine et exige une signature pour ceux téléchargés d'Internet. Relancez maintenant l'activation, puis le script de vérification :

```
Windows PowerShell
PS C:\Users\Public\Documents\M1\atelier-python> .venv\Scripts\Activate.ps1
(.venv) PS C:\Users\Public\Documents\M1\atelier-python> python verifier.py
Python 3.12.2
Interpréteur : C:\Users\Public\Documents\M1\atelier-python\.venv\Scripts\python.exe
Environnement virtuel actif : True
  requests       absent
  python-dotenv  absent
  pydantic       absent
  pytest         absent
  openai         absent
(.venv) PS C:\Users\Public\Documents\M1\atelier-python>
```

Trois changements à observer. L'invite commence par `(.venv)`. L'interpréteur est celui de `.venv`. Et toutes les bibliothèques sont absentes : l'environnement est vierge, il ne voit pas celles de la machine.

2.4 Installer les bibliothèques du projet

Environnement activé, on installe les cinq bibliothèques de l'atelier. À partir d'ici, **sous macOS aussi, la commande est `python`** : l'activation l'a rendue disponible.

Windows (PowerShell)	macOS (Terminal)
`python -m pip install requests python-dotenv pydantic pytest openai`	`python -m pip install requests python-dotenv pydantic pytest openai`

Windows PowerShell

```
(.venv) PS C:\Users\Public\Documents\M1\atelier-python> python -m pip install requests python-dotenv pydantic pytest
openai
Collecting requests
  Using cached requests-2.34.2-py3-none-any.whl.metadata (4.8 kB)
Collecting python-dotenv
  Using cached python_dotenv-1.2.3-py3-none-any.whl.metadata (29 kB)
Collecting pydantic
  Using cached pydantic-2.13.5-py3-none-any.whl.metadata (110 kB)
[ 69 lignes coupées ici pour tenir sur la page ]
Installing collected packages: urllib3, typing-extensions, truststore, sniffio, python-dotenv, pygments, pluggy, pack
aging, jiter, iniconfig, idna, h11, colorama, charset_normalizer, certifi, annotated-types, typing-inspection, reques
ts, pytest, pydantic-core, httpcore2, anyio, pydantic, httpx2, openai
Successfully installed annotated-types-0.8.0 anyio-4.15.1 certifi-2026.7.22 charset_normalizer-3.5.1 colorama-0.4.6 h
11-0.16.0 httpcore2-2.13.1 httpx2-2.13.1 idna-3.20 iniconfig-2.3.0 jiter-0.17.0 openai-3.19.2 packaging-26.3 pluggy-
1.6.0 pydantic-2.13.5 pydantic-core-2.46.5 pygments-2.21.0 pytest-9.1.1 python-dotenv-1.2.3 requests-2.34.2 sniffio-
1.3.1 truststore-0.10.4 typing-extensions-4.16.0 typing-inspection-0.4.4 urllib3-2.8.0

[notice] A new release of pip is available: 24.0 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
(.venv) PS C:\Users\Public\Documents\M1\atelier-python>
```

pip télécharge chaque bibliothèque, puis celles dont elle dépend, et termine par la liste de tout ce qu'il a installé. Le texte coupé au milieu est la longue suite des téléchargements. Si la dernière ligne n'est pas `Successfully installed`, lisez l'erreur : une connexion coupée se règle en relançant la même commande.

Bibliothèque	À quoi elle sert dans l'atelier
`requests`	envoyer des requêtes HTTP à la main (partie 9)
`python-dotenv`	lire la configuration et la clé dans un fichier `.env` (partie 8)
`pydantic`	décrire et valider la forme d'une réponse (partie 11)
`pytest`	écrire et lancer des tests automatiques (partie 12)
`openai`	le SDK officiel, qui parle à toute API au format OpenAI (partie 10)

Relancez le script de vérification :

Windows PowerShell

```
(.venv) PS C:\Users\Public\Documents\M1\atelier-python> python verifier.py
Python 3.12.2
Interpréteur : C:\Users\Public\Documents\M1\atelier-python\.venv\Scripts\python.exe
Environnement virtuel actif : True
  requests       2.34.2
  python-dotenv  1.2.3
  pydantic       2.13.5
  pytest         9.1.1
  openai         3.19.2
(.venv) PS C:\Users\Public\Documents\M1\atelier-python>
```

Les cinq bibliothèques sont là, avec leur version, et l'environnement est actif. Vos numéros de version peuvent être plus récents que ceux de la capture : c'est normal, pip prend la dernière version publiée.

2.5 Voir tout ce qui est installé

Windows (PowerShell)	macOS (Terminal)
`python -m pip list`	`python -m pip list`

Windows PowerShell

```
(.venv) PS C:\Users\Public\Documents\M1\atelier-python> python -m pip list
Package            Version
------------------ ---------
annotated-types    0.8.0
anyio              4.15.1
certifi            2026.7.22
charset-normalizer 3.5.1
colorama           0.4.6
h11                0.16.0
httpcore2          2.13.1
httpx2             2.13.1
idna               3.20
iniconfig          2.3.0
jiter              0.17.0
openai             3.19.2
packaging          26.3
pip                24.0
pluggy             1.6.0
pydantic           2.13.5
pydantic_core      2.46.5
Pygments           2.21.0
pytest             9.1.1
python-dotenv      1.2.3
requests           2.34.2
sniffio            1.3.1
truststore         0.10.4
typing_extensions  4.16.0
typing-inspection  0.4.4
urllib3            2.8.0

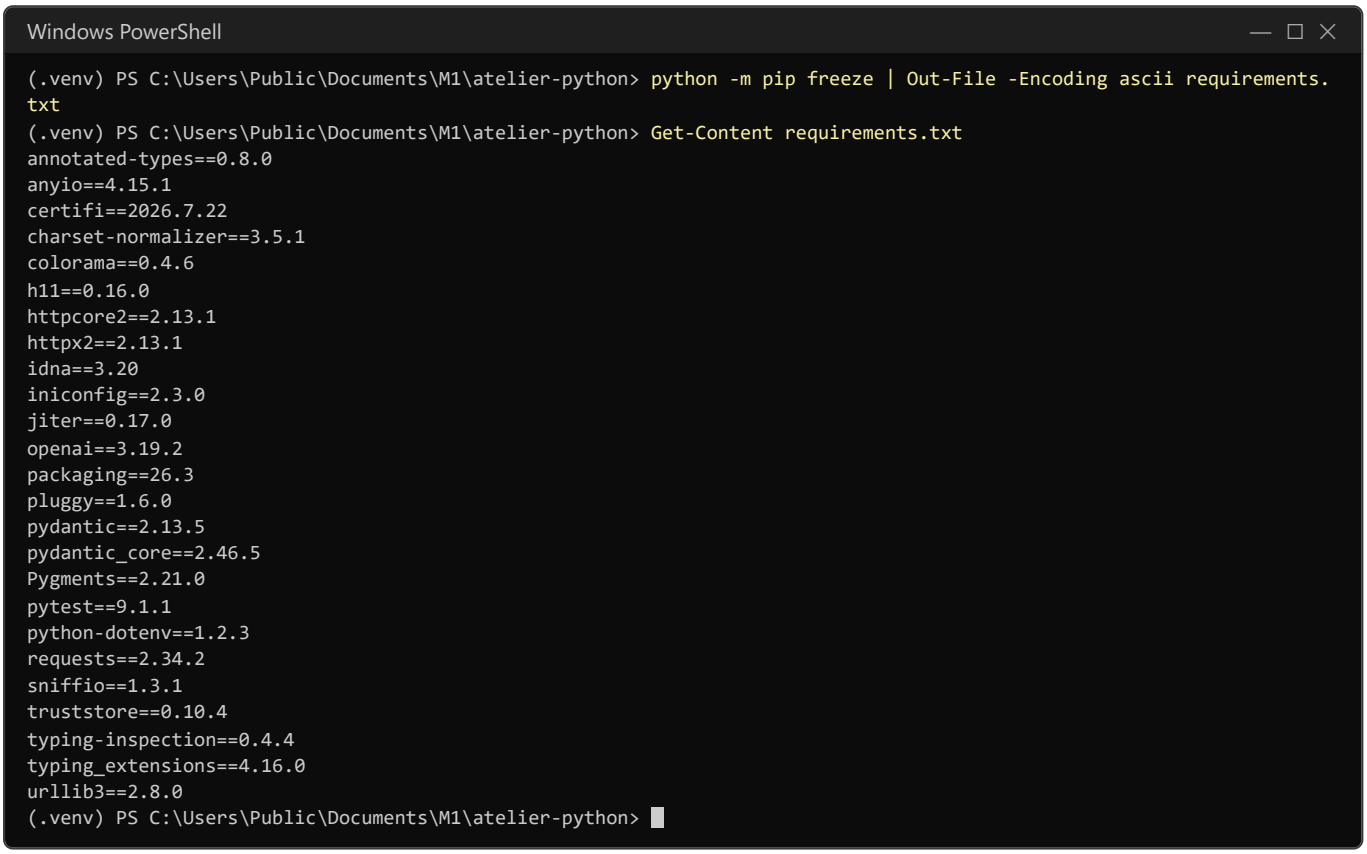
[notice] A new release of pip is available: 24.0 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
(.venv) PS C:\Users\Public\Documents\M1\atelier-python>
```

Vous avez demandé cinq bibliothèques, il y en a plus de vingt. Les autres sont leurs **dépendances** : `openai` s'appuie sur `httpx`, `pydantic` sur `pydantic_core`, et ainsi de suite. Le message final, qui annonce une version plus récente de pip, est une information : vous pouvez l'ignorer.

2.6 Figer les versions dans requirements.txt

Le dossier `.venv` ne se partage jamais : il est lourd, et il ne fonctionne que sur la machine qui l'a créé. Ce qu'on partage, c'est la **liste exacte des versions**, dans un fichier texte appelé par convention `requirements.txt`. `pip freeze` produit cette liste.

Windows (PowerShell)	macOS (Terminal)
`python -m pip freeze | Out-File -Encoding ascii requirements.txt` `Get-Content requirements.txt`	`python -m pip freeze > requirements.txt` `cat requirements.txt`



Sous Windows, on passe par `Out-File -Encoding ascii` : la redirection `>` de Windows PowerShell écrit un encodage que pip relit mal. Le format de chaque ligne est `nom==version`, avec deux signes égal.

2.7 Réinstaller depuis requirements.txt

Un collègue qui récupère votre projet crée son propre `.venv`, l'active, puis tape une seule commande pour obtenir exactement les mêmes versions que vous. Essayez-la : comme tout est déjà installé, pip se contente de le constater.

Windows (PowerShell)	macOS (Terminal)
`python -m pip install -r requirements.txt`	`python -m pip install -r requirements.txt`

Windows PowerShell

```
(.venv) PS C:\Users\Public\Documents\M1\atelier-python> python -m pip install -r requirements.txt
Requirement already satisfied: annotated-types==0.8.0 in c:\users\public\documents\m1\atelier-python\.venv\lib\site-p
ackages (from -r requirements.txt (line 1)) (0.8.0)
Requirement already satisfied: anyio==4.15.1 in c:\users\public\documents\m1\atelier-python\.venv\lib\site-packages
(from -r requirements.txt (line 2)) (4.15.1)
Requirement already satisfied: certifi==2026.7.22 in c:\users\public\documents\m1\atelier-python\.venv\lib\site-packa
ges (from -r requirements.txt (line 3)) (2026.7.22)
Requirement already satisfied: charset-normalizer==3.5.1 in c:\users\public\documents\m1\atelier-python\.venv\lib\sit
e-packages (from -r requirements.txt (line 4)) (3.5.1)
Requirement already satisfied: colorama==0.4.6 in c:\users\public\documents\m1\atelier-python\.venv\lib\site-packages
(from -r requirements.txt (line 5)) (0.4.6)
[ 19 lignes coupées ici pour tenir sur la page ]
Requirement already satisfied: urllib3==2.8.0 in c:\users\public\documents\m1\atelier-python\.venv\lib\site-packages
(from -r requirements.txt (line 25)) (2.8.0)

[notice] A new release of pip is available: 24.0 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
(.venv) PS C:\Users\Public\Documents\M1\atelier-python>
```

« Requirement already satisfied » signifie « déjà installé, dans la bonne version ». Sur une machine neuve, vous verriez à la place les téléchargements de la section 2.4.

2.8 Les réflexes à garder

- À chaque nouveau terminal, activez l'environnement avant toute chose. Si l'invite ne commence pas par `(.venv)`, il n'est pas actif.
- Dans VS Code, choisissez le même interpréteur pour l'éditeur : Ctrl+Maj+P (Cmd+Maj+P sur Mac), tapez *Python: Select Interpreter*, prenez celui qui contient `.venv`. Sans cela, l'éditeur souligne en rouge des imports qui marchent pourtant dans le terminal.
- Après chaque nouvelle installation, régénérez `requirements.txt` avec la commande de 2.6.

À vous (E2)

1. Tapez `deactivate`, relancez `python verifier.py` (`python3` sur Mac), comparez avec la capture de 2.4 : quelles lignes ont changé ? Réactivez ensuite l'environnement avec la commande de 2.3.
2. Expliquez en deux phrases pourquoi on partage `requirements.txt` et jamais le dossier `.venv`.

Partie 3. Essayer Python en direct

L'**interpréteur interactif**, qu'on appelle aussi REPL (lire, évaluer, afficher, recommencer), exécute chaque ligne dès que vous appuyez sur Entrée et affiche le résultat. C'est l'endroit où l'on teste une idée en quelques secondes avant de l'écrire dans un fichier. Lancez-le sans nom de fichier :

Windows (PowerShell)	macOS (Terminal)
`python`	`python` (environnement activé)

L'invite devient `>>>`. Tapez les lignes jaunes une par une :

Windows PowerShell

```
(.venv) PS C:\Users\Public\Documents\M1\atelier-python> python
Python 3.12.2 (tags/v3.12.2:6abddd9, Feb  6 2024, 21:26:36) [MSC v.1937 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> 2 + 3
5
>>> 7 / 2
3.5
>>> 7 // 2
3
>>> titre = "Crêpes"
>>> titre
'Crêpes'
>>> type(titre)
<class 'str'>
>>> len(titre)
6
>>> titre.upper()
'CRÊPES'
>>> temps = 30
>>> f"{titre} : {temps} minutes"
'Crêpes : 30 minutes'
>>> "30" + 5
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: can only concatenate str (not "int") to str
>>> int("30") + 5
35
>>> exit()
(.venv) PS C:\Users\Public\Documents\M1\atelier-python>
```

- `/` est la division ordinaire, elle rend toujours un nombre à virgule. `//` est la division entière.
- Une **variable** est un nom posé sur une valeur : `titre = "Crêpes"`. Taper le nom seul affiche sa valeur. `type()` donne sa nature, ici `str`, une chaîne de caractères.
- Une chaîne a des **méthodes**, des fonctions qu'on appelle avec un point : `titre.upper()`.
- La **f-string**, une chaîne précédée de `f`, insère des valeurs entre accolades. C'est la façon normale de fabriquer un texte en Python.
- `"30" + 5` échoue : Python ne mélange pas un texte et un nombre. La dernière ligne du message, `TypeError`, dit exactement pourquoi. `int("30")` convertit le texte en nombre, et l'addition passe.
- `exit()` quitte l'interpréteur et rend la main à PowerShell ou au Terminal.

À vous (E3)

Relancez l'interpréteur et prévoyez le résultat de chaque ligne **avant** de l'exécuter : `10 // 3`, `10 % 3`, `"crêpes".capitalize()`, `str(30) + " min"`, `len("oeufs")`. Notez vos prévisions et les vrais résultats dans `reponses.txt`.

Partie 4. Les bases du langage, sur une recette

Vous construisez le fichier `bases.py` en cinq étapes. À chaque étape, vous ajoutez les lignes vertes à la fin du fichier, vous enregistrez, et vous relancez le même fichier avec la même commande :

Windows (PowerShell)	macOS (Terminal)
`python bases.py`	`python bases.py`

Astuce : dans le terminal, la flèche du haut rappelle la commande précédente. Flèche haut puis Entrée suffit pour relancer.

4.1 Variables et types

Quatre types de base suffisent à décrire une recette : le texte (`str`), l'entier (`int`), le nombre à virgule (`float` , avec un point et non une virgule) et le booléen (`bool` , qui vaut `True` ou `False` , avec une majuscule). Créez `bases.py` :

bases.py

```
# bases.py : les types de base de Python, sur une recette
titre = "Crêpes"
temps_min = 30
note = 4.5
vegetarien = True

print(titre, "se prépare en", temps_min, "minutes.")
print(type(titre), type(temps_min), type(note), type(vegetarien))
print(f"{titre} : note {note} sur 5, végétarien : {vegetarien}")
```

Windows PowerShell

```
(.venv) PS C:\Users\Public\Documents\M1\atelier-python> python bases.py
Crêpes se prépare en 30 minutes.
<class 'str'> <class 'int'> <class 'float'> <class 'bool'>
Crêpes : note 4.5 sur 5, végétarien : True
(.venv) PS C:\Users\Public\Documents\M1\atelier-python>
```

`print()` accepte plusieurs valeurs séparées par des virgules et les affiche séparées par un espace. La deuxième ligne confirme le type de chaque variable. La troisième est une f-string.

4.2 Les listes

Une **liste** est une suite ordonnée de valeurs, entre crochets. On compte à partir de 0 : `[0]` est le premier élément, `[-1]` le dernier. `[:2]` est une **tranche**, les éléments d'indice 0 et 1. Ajoutez à la fin du fichier :

bases.py

```
    (lignes 1 à 7 inchangées)
8   print(type(titre), type(temps_min), type(note), type(vegetarien))
9   print(f"{titre} : note {note} sur 5, végétarien : {vegetarien}")
10
11  ingredients = ["farine", "oeufs", "lait", "beurre"]
12  print("Nombre d'ingrédients :", len(ingredients))
13  print("Premier :", ingredients[0], "| dernier :", ingredients[-1])
14  ingredients.append("sucre")
15  print("Après ajout :", ingredients)
16  print("Les deux premiers :", ingredients[:2])
17  print("Du lait dedans ?", "lait" in ingredients)
```

Windows PowerShell

```
(.venv) PS C:\Users\Public\Documents\M1\atelier-python> python bases.py
Crêpes se prépare en 30 minutes.
<class 'str'> <class 'int'> <class 'float'> <class 'bool'>
Crêpes : note 4.5 sur 5, végétarien : True
Nombre d'ingrédients : 4
Premier : farine | dernier : beurre
Après ajout : ['farine', 'oeufs', 'lait', 'beurre', 'sucre']
Les deux premiers : ['farine', 'oeufs']
Du lait dedans ? True
(.venv) PS C:\Users\Public\Documents\M1\atelier-python>
```

`append` modifie la liste sur place : elle passe de quatre à cinq ingrédients. L'opérateur `in` répond par `True` ou `False`.

4.3 Les dictionnaires

Un **dictionnaire** associe des **clés** à des valeurs, entre accolades. C'est la structure que vous manipulerez le plus en IA, parce que c'est exactement la forme d'un objet JSON, et donc d'une réponse d'API.

bases.py

```
(lignes 1 à 15 inchangées)
print("Les deux premiers :", ingredients[:2])
print("Du lait dedans ?", "lait" in ingredients)

recette = {"titre": titre, "temps_min": temps_min, "ingredients": ingredients}
print("Clés :", list(recette.keys()))
print("Temps lu dans le dictionnaire :", recette["temps_min"])
recette["portions"] = 4
print("Portions :", recette["portions"])
print("Difficulté :", recette.get("difficulte", "non renseignée"))
```

Windows PowerShell

```
(.venv) PS C:\Users\Public\Documents\M1\atelier-python> python bases.py
Crêpes se prépare en 30 minutes.
<class 'str'> <class 'int'> <class 'float'> <class 'bool'>
Crêpes : note 4.5 sur 5, végétarien : True
Nombre d'ingrédients : 4
Premier : farine | dernier : beurre
Après ajout : ['farine', 'oeufs', 'lait', 'beurre', 'sucre']
Les deux premiers : ['farine', 'oeufs']
Du lait dedans ? True
Clés : ['titre', 'temps_min', 'ingredients']
Temps lu dans le dictionnaire : 30
Portions : 4
Difficulté : non renseignée
(.venv) PS C:\Users\Public\Documents\M1\atelier-python>
```

- `recette["temps_min"]` lit la valeur d'une clé. Si la clé n'existe pas, c'est une erreur.
- `recette["portions"] = 4` ajoute une clé, ou remplace sa valeur si elle existe.
- `recette.get("difficulte", "non renseignée")` lit une clé sans risque : si elle manque, on obtient la valeur par défaut. C'est le bon réflexe face à des données qu'on ne maîtrise pas.

4.4 Conditions et boucles

`if`, `elif`, `else` choisissent un chemin selon une condition. `for` répète un bloc pour chaque élément d'une liste. En Python, **c'est l'indentation qui délimite les blocs** : quatre espaces après chaque ligne qui se termine par deux points. VS Code les insère pour vous quand vous appuyez sur Entrée.

bases.py

```
    (lignes 1 à 22 inchangées)
23  print("Portions :", recette["portions"])
24  print("Difficulté :", recette.get("difficulte", "non renseignée"))
25
26  if temps_min <= 20:
27      rythme = "express"
28  elif temps_min <= 45:
29      rythme = "rapide"
30  else:
31      rythme = "long"
32  print("Rythme :", rythme)
33
34  for numero, ingredient in enumerate(ingredients, start=1):
35      print(f"  {numero}. {ingredient}")
```

Windows PowerShell

```
(.venv) PS C:\Users\Public\Documents\M1\atelier-python> python bases.py
Crêpes se prépare en 30 minutes.
<class 'str'> <class 'int'> <class 'float'> <class 'bool'>
Crêpes : note 4.5 sur 5, végétarien : True
Nombre d'ingrédients : 4
Premier : farine | dernier : beurre
Après ajout : ['farine', 'oeufs', 'lait', 'beurre', 'sucre']
Les deux premiers : ['farine', 'oeufs']
Du lait dedans ? True
Clés : ['titre', 'temps_min', 'ingredients']
Temps lu dans le dictionnaire : 30
Portions : 4
Difficulté : non renseignée
Rythme : rapide
  1. farine
  2. oeufs
  3. lait
  4. beurre
  5. sucre
(.venv) PS C:\Users\Public\Documents\M1\atelier-python>
```

30 minutes n'est pas inférieur ou égal à 20, mais l'est à 45 : le rythme vaut « rapide ». `enumerate(..., start=1)` fournit à la fois un numéro et l'élément, ce qui évite de tenir un compteur à la main.

4.5 Les fonctions

Une **fonction** donne un nom à un bloc de code réutilisable. Elle se déclare avec `def`, reçoit des **paramètres**, et rend un résultat avec `return`. Un paramètre peut avoir une valeur par défaut, comme `portions=4`. Le texte entre triples guillemets, juste sous `def`, est la **docstring** : il dit ce que fait la fonction, et VS Code l'affiche quand on survole son nom.

bases.py

```
    (lignes 1 à 33 inchangées)
34  for numero, ingredient in enumerate(ingredients, start=1):
35      print(f"  {numero}. {ingredient}")
36
37
38  def resumer(titre, temps_min, portions=4):
39      """Rend une ligne de présentation d'une recette."""
40      return f"{titre} ({temps_min} min, {portions} portions)"
41
42
43  print(resumer("Crêpes", 30))
44  print(resumer("Taboulé", 25, portions=6))
```

Windows PowerShell

```
(.venv) PS C:\Users\Public\Documents\M1\atelier-python> python bases.py
Crêpes se prépare en 30 minutes.
<class 'str'> <class 'int'> <class 'float'> <class 'bool'>
Crêpes : note 4.5 sur 5, végétarien : True
Nombre d'ingrédients : 4
Premier : farine | dernier : beurre
Après ajout : ['farine', 'oeufs', 'lait', 'beurre', 'sucre']
Les deux premiers : ['farine', 'oeufs']
Du lait dedans ? True
Clés : ['titre', 'temps_min', 'ingredients']
Temps lu dans le dictionnaire : 30
Portions : 4
Difficulté : non renseignée
Rythme : rapide
  1. farine
  2. oeufs
  3. lait
  4. beurre
  5. sucre
Crêpes (30 min, 4 portions)
Taboulé (25 min, 6 portions)
(.venv) PS C:\Users\Public\Documents\M1\atelier-python>
```

Le premier appel utilise la valeur par défaut, le second la remplace en nommant le paramètre. Nommer les paramètres optionnels rend l'appel lisible sans relire la définition. Notez aussi qu'une fonction qui **rend** un texte n'affiche rien : c'est le `print` autour de l'appel qui l'affiche.

À vous (E4 et E5)

E4. À la fin de `bases.py`, écrivez une fonction `est_rapide(temps_min, seuil=20)` qui rend `True` si la recette se prépare en `seuil` minutes ou moins. Affichez le résultat de `est_rapide(15)`, `est_rapide(30)` et `est_rapide(30, seuil=45)`.

E5. Faites en sorte que la ligne « Difficulté » affiche `facile` au lieu de « non renseignée », en ne modifiant qu'une seule ligne du fichier. Laquelle avez-vous changée, et pourquoi la ligne avec `get` n'a-t-elle pas eu besoin d'être touchée ?

Partie 5. Lire un fichier JSON et en tirer des chiffres

Les données d'un projet d'IA arrivent presque toujours en JSON : fichiers de configuration, jeux de test, réponses d'API. Le module `json` de Python transforme ce texte en listes et dictionnaires, que vous savez maintenant manipuler. Ouvrez `donnees/recettes.json` dans VS Code pour voir sa forme : une liste de quinze objets, chacun avec un titre, une catégorie, un temps, des ingrédients et des étiquettes (`tags`).

5.1 Ouvrir et charger le fichier

Créez `explorer.py` . Deux notions nouvelles : `pathlib.Path` construit un chemin avec l'opérateur `/` et choisit le bon séparateur selon le système ; `with open(...) as fichier` ouvre le fichier et garantit sa fermeture à la fin du bloc, même en cas d'erreur.

explorer.py

```
# explorer.py : lire le fichier de recettes et en tirer des chiffres
import json
from pathlib import Path

chemin = Path("donnees") / "recettes.json"
print("Chemin :", chemin)
print("Le fichier existe :", chemin.exists())

with open(chemin, encoding="utf-8") as fichier:
    recettes = json.load(fichier)

print("Type lu :", type(recettes).__name__)
print("Nombre de recettes :", len(recettes))
print("Première recette :", recettes[0])
```

Windows (PowerShell)	macOS (Terminal)
`python explorer.py`	`python explorer.py`

Windows PowerShell

```
(.venv) PS C:\Users\Public\Documents\M1\atelier-python> python explorer.py
Chemin : donnees\recettes.json
Le fichier existe : True
Type lu : list
Nombre de recettes : 15
Première recette : {'id': 1, 'titre': 'Ratatouille', 'categorie': 'plat', 'temps_min': 75, 'ingredients': ['courgett
e', 'aubergine', 'poivron', 'tomate', 'oignon', "huile d'olive"], 'tags': ['vegetarien', 'sans_gluten']}
(.venv) PS C:\Users\Public\Documents\M1\atelier-python>
```

`json.load` a rendu une `list` de 15 éléments, et chaque élément est un dictionnaire. Sur Mac, le chemin s'affiche `donnees/recettes.json` : c'est `pathlib` qui s'adapte, votre code ne change pas.

5.2 Additionner dans une boucle

Le calcul d'une moyenne à la main : une variable qui accumule, une boucle qui ajoute. Ajoutez :

explorer.py

```
    (lignes 1 à 12 inchangées)
13  print("Nombre de recettes :", len(recettes))
14  print("Première recette :", recettes[0])
15
16  total = 0
17  for r in recettes:
18      total = total + r["temps_min"]
19  print(f"Temps moyen : {total / len(recettes):.1f} minutes")
```

Windows PowerShell

```
(.venv) PS C:\Users\Public\Documents\M1\atelier-python> python explorer.py
Chemin : donnees\recettes.json
Le fichier existe : True
Type lu : list
Nombre de recettes : 15
Première recette : {'id': 1, 'titre': 'Ratatouille', 'categorie': 'plat', 'temps_min': 75, 'ingredients': ['courgett
e', 'aubergine', 'poivron', 'tomate', 'oignon', "huile d'olive"], 'tags': ['vegetarien', 'sans_gluten']}
Temps moyen : 43.0 minutes
(.venv) PS C:\Users\Public\Documents\M1\atelier-python>
```

`:.1f` , dans l'accolade de la f-string, formate le nombre avec un seul chiffre après la virgule.

5.3 Les compréhensions de liste

Une **compréhension de liste** fabrique une nouvelle liste en une ligne : `[expression for element in liste if condition]`. Elle remplace la boucle avec `append` dans l'immense majorité des cas, et vous en lirez partout dans du code d'IA.

explorer.py

```
(lignes 1 à 17 inchangées)
    total = total + r["temps_min"]
print(f"Temps moyen : {total / len(recettes):.1f} minutes")

temps = [r["temps_min"] for r in recettes]
print("Plus court :", min(temps), "min, plus long :", max(temps), "min")

rapides = [r["titre"] for r in recettes if r["temps_min"] <= 20]
print("Prêtes en 20 minutes ou moins :", rapides)

vegetariennes = [r["titre"] for r in recettes if "vegetarien" in r["tags"]]
print("Végétariennes :", len(vegetariennes), vegetariennes)
```

Windows PowerShell

```
(.venv) PS C:\Users\Public\Documents\M1\atelier-python> python explorer.py
Chemin : donnees\recettes.json
Le fichier existe : True
Type lu : list
Nombre de recettes : 15
Première recette : {'id': 1, 'titre': 'Ratatouille', 'categorie': 'plat', 'temps_min': 75, 'ingredients': ['courgett
e', 'aubergine', 'poivron', 'tomate', 'oignon', "huile d'olive"], 'tags': ['vegetarien', 'sans_gluten']}
Temps moyen : 43.0 minutes
Plus court : 15 min, plus long : 90 min
Prêtes en 20 minutes ou moins : ['Omelette aux champignons', 'Croque-monsieur', 'Pâtes carbonara', 'Mousse au chocola
t']
Végétariennes : 8 ['Ratatouille', 'Omelette aux champignons', 'Gratin dauphinois', 'Soupe de potiron', 'Risotto aux c
hampignons', 'Crêpes', 'Mousse au chocolat', 'Tarte aux pommes']
(.venv) PS C:\Users\Public\Documents\M1\atelier-python>
```

Quatre recettes sont prêtes en 20 minutes ou moins. Le compte des végétariennes donne 8. Regardez pourtant les étiquettes dans le fichier JSON : certaines recettes portent `vegan` sans porter `vegetarien`, et une recette vegan est aussi végétarienne. Le code est juste, la question était mal posée.

5.4 Corriger le filtre, compter, trier

On corrige la condition des végétariennes (lignes barrée et ajoutées), puis on compte les recettes par catégorie avec un dictionnaire, et on trie avec `sorted`. Le paramètre `key` de `sorted` reçoit une fonction qui dit selon quoi trier : ici le temps.

explorer.py

```
    (lignes 1 à 24 inchangées)
25  print("Prêtes en 20 minutes ou moins :", rapides)
26
    vegetariennes = [r["titre"] for r in recettes if "vegetarien" in r["tags"]]
27  vegetariennes = [r["titre"] for r in recettes
28                   if "vegetarien" in r["tags"] or "vegan" in r["tags"]]
29  print("Végétariennes :", len(vegetariennes), vegetariennes)
30
31  par_categorie = {}
32  for r in recettes:
33      categorie = r["categorie"]
34      par_categorie[categorie] = par_categorie.get(categorie, 0) + 1
35  print("Par catégorie :", par_categorie)
36
37
38  def duree(recette):
39      return recette["temps_min"]
40
41
42  triees = sorted(recettes, key=duree)
43  print("Les trois plus rapides :", [r["titre"] for r in triees[:3]])
```

Windows PowerShell

```
(.venv) PS C:\Users\Public\Documents\M1\atelier-python> python explorer.py
Chemin : donnees\recettes.json
Le fichier existe : True
Type lu : list
Nombre de recettes : 15
Première recette : {'id': 1, 'titre': 'Ratatouille', 'categorie': 'plat', 'temps_min': 75, 'ingredients': ['courgett
e', 'aubergine', 'poivron', 'tomate', 'oignon', "huile d'olive"], 'tags': ['vegetarien', 'sans_gluten']}
Temps moyen : 43.0 minutes
Plus court : 15 min, plus long : 90 min
Prêtes en 20 minutes ou moins : ['Omelette aux champignons', 'Croque-monsieur', 'Pâtes carbonara', 'Mousse au chocola
t']
Végétariennes : 11 ['Ratatouille', 'Omelette aux champignons', 'Gratin dauphinois', 'Soupe de potiron', 'Salade de le
ntilles', 'Risotto aux champignons', 'Chili sin carne', 'Taboulé', 'Crêpes', 'Mousse au chocolat', 'Tarte aux pomme
s']
Par catégorie : {'plat': 8, 'accompagnement': 1, 'entree': 3, 'dessert': 3}
Les trois plus rapides : ['Omelette aux champignons', 'Croque-monsieur', 'Pâtes carbonara']
(.venv) PS C:\Users\Public\Documents\M1\atelier-python>
```

Les végétariennes passent de 8 à 11. `par_categorie.get(categorie, 0) + 1` est le motif classique du comptage : on lit la valeur actuelle, zéro si la clé n'existe pas encore, et on ajoute un. La fonction `duree` est passée à `sorted` sans parenthèses : on donne la fonction elle-même, pas son résultat.

À vous (E6)

Dans `explorer.py`, affichez : la liste des recettes portant l'étiquette `sans_gluten` et leur nombre ; les desserts triés du plus rapide au plus long, avec leur temps ; le titre de la recette la plus longue, en une ligne, avec `max` et son paramètre `key`.

Partie 6. Ranger son code dans un module

Un **module** est simplement un fichier `.py` dont un autre fichier importe les fonctions. On y range le code qu'on veut réutiliser. Le fichier `recettes.py` que vous allez écrire servira jusqu'à la fin de l'atelier : les tests, l'évaluation et l'assistant l'importent tous. Voici sa première moitié :

recettes.py

```
"""Lire le fichier de recettes et y chercher."""
import json
from pathlib import Path

FICHIER = Path(__file__).parent / "donnees" / "recettes.json"


def charger(chemin=FICHIER):
    """Rend la liste des recettes lue dans le fichier JSON."""
    with open(chemin, encoding="utf-8") as f:
        return json.load(f)


def filtrer_par_temps(recettes, maxi):
    """Rend les recettes prêtes en maxi minutes ou moins."""
    return [r for r in recettes if r["temps_min"] <= maxi]

```

- `Path(__file__).parent` est le dossier qui contient `recettes.py`. Le chemin du fichier JSON est donc calculé à partir du module, et non du dossier où se trouve votre terminal. Le module marchera d'où qu'on l'appelle.
- Le nom `FICHIER` en majuscules signale une **constante** : une valeur qu'on ne modifie pas pendant l'exécution.
- `charger(chemin=FICHIER)` charge le fichier par défaut, mais accepte un autre chemin. Cette souplesse servira aux tests et à la partie 7.

La seconde moitié, à taper à la suite dans le même fichier :

recettes.py

```
(lignes 1 à 18 inchangées)
def chercher_ingredient(recettes, ingredient):
    """Rend les recettes dont un ingrédient contient le mot cherché."""
    cherche = ingredient.lower()
    trouvees = []
    for r in recettes:
        for i in r["ingredients"]:
            if cherche in i:
                trouvees.append(r)
                break
    return trouvees


def resumer(recette):
    """Rend une ligne courte : titre et temps."""
    return f"{recette['titre']} ({recette['temps_min']} min)"


if __name__ == "__main__":
    toutes = charger()
    print(len(toutes), "recettes chargées")
    for r in filtrer_par_temps(toutes, 20):
        print("  ", resumer(r))
    print("Avec des champignons :", [resumer(r) for r in chercher_ingredient(toutes, "Champignons")])
```

- `break` sort de la boucle intérieure dès qu'un ingrédient correspond, pour ne pas ajouter deux fois la même recette.
- Le bloc `if __name__ == "__main__":` ne s'exécute que si l'on lance `recettes.py` directement. Quand un autre fichier l'importe, il est ignoré. C'est l'endroit pour une petite démonstration du module.

6.1 Lancer le module directement

Windows (PowerShell)	macOS (Terminal)
`python recettes.py`	`python recettes.py`

Windows PowerShell

```
(.venv) PS C:\Users\Public\Documents\M1\atelier-python> python recettes.py
15 recettes chargées
   Omelette aux champignons (15 min)
   Croque-monsieur (20 min)
   Pâtes carbonara (20 min)
   Mousse au chocolat (20 min)
Avec des champignons : ['Omelette aux champignons (15 min)', 'Risotto aux champignons (35 min)']
(.venv) PS C:\Users\Public\Documents\M1\atelier-python>
```

Quatre recettes rapides, puis deux avec des champignons. Regardez l'appel : on cherche « Champignons » avec une majuscule, et la fonction met la recherche en minuscules avec `lower()`. Ce détail cache un défaut que les tests de la partie 12 vont révéler.

6.2 Importer le module depuis un autre fichier

Créez `utiliser.py`. `import recettes` charge le fichier `recettes.py` du même dossier ; on appelle ensuite ses fonctions en les préfixant du nom du module.

utiliser.py

```
# utiliser.py : se servir du module recettes depuis un autre fichier
import recettes

toutes = recettes.charger()
choix = recettes.chercher_ingredient(toutes, "oeufs")
print(len(choix), "recettes avec des oeufs :")
for r in choix:
    print("  ", recettes.resumer(r))
```

Windows PowerShell

```
(.venv) PS C:\Users\Public\Documents\M1\atelier-python> python utiliser.py
5 recettes avec des oeufs :
   Quiche lorraine (55 min)
   Omelette aux champignons (15 min)
   Pâtes carbonara (20 min)
   Crêpes (30 min)
   Mousse au chocolat (20 min)
(.venv) PS C:\Users\Public\Documents\M1\atelier-python>
```

La démonstration du bloc `__main__` de `recettes.py` ne s'est pas affichée : c'est la preuve qu'un import ne l'exécute pas. Seconde forme d'import, qui fait entrer des noms précis dans le fichier et dispense du préfixe :

utiliser.py

```
    (lignes 1 à 6 inchangées)
for r in choix:
    print("  ", recettes.resumer(r))

from recettes import filtrer_par_temps, resumer

for r in filtrer_par_temps(choix, 20):
    print("Rapide et avec des oeufs :", resumer(r))
```

Windows PowerShell

```
(.venv) PS C:\Users\Public\Documents\M1\atelier-python> python utiliser.py
5 recettes avec des oeufs :
   Quiche lorraine (55 min)
   Omelette aux champignons (15 min)
   Pâtes carbonara (20 min)
   Crêpes (30 min)
   Mousse au chocolat (20 min)
Rapide et avec des oeufs : Omelette aux champignons (15 min)
Rapide et avec des oeufs : Pâtes carbonara (20 min)
Rapide et avec des oeufs : Mousse au chocolat (20 min)
(.venv) PS C:\Users\Public\Documents\M1\atelier-python>
```

6.3 Ce que Python a laissé dans le dossier

Windows (PowerShell)	macOS (Terminal)
`Get-ChildItem -Name` `Get-ChildItem __pycache__ -Name`	`ls` `ls __pycache__`



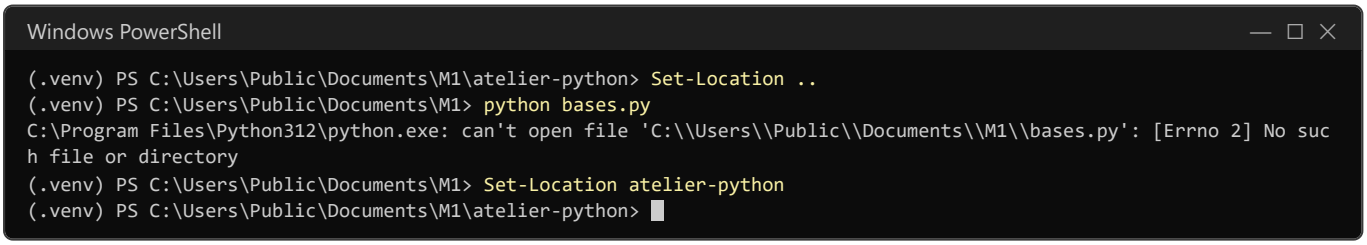
Au premier import, Python a compilé `recettes.py` dans le dossier `__pycache__`, pour aller plus vite la fois suivante. Ce dossier se recrée seul, ne se modifie jamais à la main, et ne se partage pas : il ira dans le fichier `.gitignore` à la partie 8.

6.4 L'erreur du mauvais dossier

Faites exprès l'erreur la plus courante : remonter d'un dossier, lancer un script, et revenir.

Windows (PowerShell)	macOS (Terminal)

`Set-Location ..`
`python bases.py`
`Set-Location atelier-python`
	`cd ..`
	`python bases.py`
	`cd atelier-python`



« can't open file » avec « No such file or directory » veut toujours dire la même chose : le terminal n'est pas dans le dossier qui contient le fichier. Lisez le chemin affiché dans le message : il montre où Python a cherché. Le remède est de revenir dans `atelier-python`, comme la troisième commande.

À vous (E7)

Ajoutez au module `recettes.py` une fonction `par_categorie(recettes, categorie)` qui rend les recettes d'une catégorie donnée, avec une compréhension de liste et une docstring. Placez-la au-dessus du bloc `__main__`. Appelez-la depuis `utiliser.py` pour afficher les entrées (`"entree"`, sans accent dans le fichier), avec `resumer`.

Partie 7. Lire une erreur, la rattraper, en lever une

Un programme qui appelle un modèle rencontre des erreurs à chaque étape : fichier absent, clé manquante dans une réponse, JSON mal formé, serveur injoignable. Savoir lire le message et décider quoi faire est la moitié du métier.

7.1 Lire une trace d'erreur

Créez `erreurs.py` avec une faute volontaire dans le nom du fichier (`recette.json` au lieu de `recettes.json`) :

erreurs.py

```python
# erreurs.py : provoquer une erreur, lire le message, puis la rattraper
import recettes

toutes = recettes.charger("donnees/recette.json")
print(len(toutes), "recettes")
```

Windows PowerShell

```
(.venv) PS C:\Users\Public\Documents\M1\atelier-python> python erreurs.py
Traceback (most recent call last):
  File "C:\Users\Public\Documents\M1\atelier-python\erreurs.py", line 4, in <module>
    toutes = recettes.charger("donnees/recette.json")
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\Public\Documents\M1\atelier-python\recettes.py", line 10, in charger
    with open(chemin, encoding="utf-8") as f:
         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
FileNotFoundError: [Errno 2] No such file or directory: 'donnees/recette.json'
(.venv) PS C:\Users\Public\Documents\M1\atelier-python>
```

Une trace se lit **de bas en haut**. La dernière ligne donne le type d'erreur, `FileNotFoundError`, et le détail : le fichier cherché. Les lignes du dessus remontent le chemin des appels : la ligne 4 d'`erreurs.py` a appelé `charger`, dont la ligne 10 a appelé `open`. Le fichier que vous devez corriger est le vôtre, pas celui de Python.

7.2 Rattraper une erreur avec try / except

`try` essaie un bloc ; si une erreur du type indiqué survient, Python saute dans le bloc `except` correspondant au lieu de s'arrêter. `as err` donne accès aux détails de l'erreur. Remplacez tout le contenu d'`erreurs.py` :

erreurs.py

```python
# erreurs.py : provoquer une erreur, lire le message, puis la rattraper
import json
import recettes

~~toutes = recettes.charger("donnees/recette.json")~~
try:
    toutes = recettes.charger("donnees/recette.json")
except FileNotFoundError as err:
    print("Fichier introuvable :", err.filename)
    toutes = recettes.charger()
print(len(toutes), "recettes")

premiere = toutes[0]
try:
    print(premiere["difficulte"])
except KeyError as err:
    print("Clé absente :", err)

texte_recu = '{"intention": "conservation", "confiance": 0.9,}'
try:
    donnees = json.loads(texte_recu)
except json.JSONDecodeError as err:
    print("JSON invalide :", err)
```

```
Windows PowerShell
(.venv) PS C:\Users\Public\Documents\M1\atelier-python> python erreurs.py
Fichier introuvable : donnees/recette.json
15 recettes
Clé absente : 'difficulte'
JSON invalide : Expecting property name enclosed in double quotes: line 1 column 48 (char 47)
(.venv) PS C:\Users\Public\Documents\M1\atelier-python>
```

Trois erreurs, trois rattrapages, et le programme va au bout :

- le fichier absent : on le signale, puis on charge le bon fichier ;
- la clé absente : `KeyError`, la même erreur que vous auriez en lisant une clé qu'une API n'a pas renvoyée ;
- le JSON invalide : la virgule avant l'accolade fermante est interdite en JSON. Le message donne la position exacte, caractère 47. **Les modèles de langage produisent souvent ce genre de JSON presque juste** : la partie 11 y revient.

Ne rattrapez que les erreurs que vous savez traiter, en nommant leur type. Un `except:` seul, sans type, avale toutes les erreurs, y compris vos fautes de frappe, et rend le programme impossible à déboguer.

7.3 Lever une erreur soi-même

Une fonction qui reçoit une valeur absurde doit le dire, plutôt que de rendre un résultat faux. On **lève** une erreur avec `raise`. Dans `recettes.py`, ajoutez deux lignes au début de `filtrer_par_temps` :

recettes.py

```
(lignes 1 à 13 inchangées)
def filtrer_par_temps(recettes, maxi):
    """Rend les recettes prêtes en maxi minutes ou moins."""
    if maxi <= 0:
        raise ValueError(f"maxi doit être positif, reçu {maxi}")
    return [r for r in recettes if r["temps_min"] <= maxi]

(lignes 20 à 43 inchangées)
```

Puis ajoutez à la fin d'`erreurs.py` un appel avec un temps négatif, rattrapé :

erreurs.py

```
(lignes 1 à 20 inchangées)
except json.JSONDecodeError as err:
    print("JSON invalide :", err)

try:
    recettes.filtrer_par_temps(toutes, -5)
except ValueError as err:
    print("Valeur refusée :", err)
```

Windows PowerShell

```
(.venv) PS C:\Users\Public\Documents\M1\atelier-python> python erreurs.py
Fichier introuvable : donnees/recette.json
15 recettes
Clé absente : 'difficulte'
JSON invalide : Expecting property name enclosed in double quotes: line 1 column 48 (char 47)
Valeur refusée : maxi doit être positif, reçu -5
(.venv) PS C:\Users\Public\Documents\M1\atelier-python>
```

`ValueError` est le type prévu pour « la valeur a le bon type mais n'a pas de sens ».

À vous (E8)

Dans l'interpréteur, tapez `int("trente")`. Notez le type d'erreur. Écrivez ensuite, dans `erreurs.py`, un bloc `try / except` qui convertit ce texte et affiche « Nombre illisible : » suivi du message de l'erreur.

Partie 8. Sortir la configuration et la clé du code

Pour appeler un modèle, il faut une adresse, un nom de modèle et une **clé d'API**, qui vaut une carte bancaire : qui la détient consomme à vos frais. Une clé écrite dans le code finit tôt ou tard sur GitHub, où des robots la récupèrent en quelques minutes. La règle est absolue : **la clé vit dans un fichier `.env`, qui ne quitte jamais votre machine**.

8.1 Le fichier .env

Créez à la racine du projet un fichier nommé exactement `.env`, avec le point au début et sans extension. Une ligne par variable, `NOM=valeur`, sans espace ni guillemet :

.env

```
MODELE_URL=http://127.0.0.1:8000/v1
MODELE_CLE=sk-atelier-local
MODELE_NOM=modele-local
```

La clé `sk-atelier-local` ne vaut que pour le modèle local de l'atelier, qui accepte toute clé commençant par `sk-`. Avec un vrai fournisseur, vous colleriez ici la clé générée dans votre espace client.

8.2 Le modèle .env.exemple

Comment un collègue sait-il quelles variables remplir, si `.env` n'est jamais partagé ? On partage à la place un modèle, sans les valeurs secrètes. Créez `.env.exemple` :

.env.exemple

```
# Copiez ce fichier sous le nom .env puis remplissez les valeurs.
MODELE_URL=http://127.0.0.1:8000/v1
MODELE_CLE=
MODELE_NOM=modele-local
```

8.3 Le fichier .gitignore

Git, l'outil de gestion de versions, lit le fichier `.gitignore` pour savoir quels fichiers ne jamais suivre. Créez-le avec quatre lignes : le secret, l'environnement virtuel, le cache de Python, et le fichier de résultats de la partie 13, qui se régénère.

.gitignore

```
.env
.venv/
__pycache__/
resultats.csv
```

8.4 Lire la configuration en Python

Créez `config.py`. `load_dotenv()`, de la bibliothèque `python-dotenv`, lit le fichier `.env` et range ses lignes dans les **variables d'environnement** du programme. `os.getenv("NOM", défaut)` les relit, avec une valeur par défaut si la variable n'existe pas.

config.py

```
"""Lit la configuration dans le fichier .env, jamais dans le code."""
import os
from dotenv import load_dotenv

load_dotenv()

URL = os.getenv("MODELE_URL", "http://127.0.0.1:8000/v1")
CLE = os.getenv("MODELE_CLE")
NOM = os.getenv("MODELE_NOM", "modele-local")

if __name__ == "__main__":
    print("URL :", URL)
    print("Modèle :", NOM)
    print("Clé :", CLE[:6] + "..." if CLE else "absente")
```

Windows (PowerShell)	macOS (Terminal)
`python config.py`	`python config.py`

```
Windows PowerShell
(.venv) PS C:\Users\Public\Documents\M1\atelier-python> python config.py
URL : http://127.0.0.1:8000/v1
Modèle : modele-local
Clé : sk-ate...
(.venv) PS C:\Users\Public\Documents\M1\atelier-python>
```

La clé n'est affichée qu'en partie : `CLE[:6]` garde les six premiers caractères. On n'affiche jamais une clé entière, même dans un terminal, parce que les captures d'écran circulent. Les autres fichiers du projet feront `import config` et liront `config.URL`, `config.CLE` et `config.NOM`.

8.5 Une variable d'environnement l'emporte sur .env

Par défaut, `load_dotenv()` ne remplace pas une variable d'environnement déjà définie. On peut donc changer un réglage le temps d'un essai, sans toucher au fichier :

Windows (PowerShell)	macOS (Terminal)

`$env:MODELE_NOM = "autre-modele"`
`python config.py`
`Remove-Item Env:MODELE_NOM`
`python config.py`
	`export MODELE_NOM="autre-modele"`
	`python config.py`
	`unset MODELE_NOM`
	`python config.py`

```
Windows PowerShell
(.venv) PS C:\Users\Public\Documents\M1\atelier-python> $env:MODELE_NOM = "autre-modele"
(.venv) PS C:\Users\Public\Documents\M1\atelier-python> python config.py
URL : http://127.0.0.1:8000/v1
Modèle : autre-modele
Clé : sk-ate...
(.venv) PS C:\Users\Public\Documents\M1\atelier-python> Remove-Item Env:MODELE_NOM
(.venv) PS C:\Users\Public\Documents\M1\atelier-python> python config.py
URL : http://127.0.0.1:8000/v1
Modèle : modele-local
Clé : sk-ate...
(.venv) PS C:\Users\Public\Documents\M1\atelier-python>
```

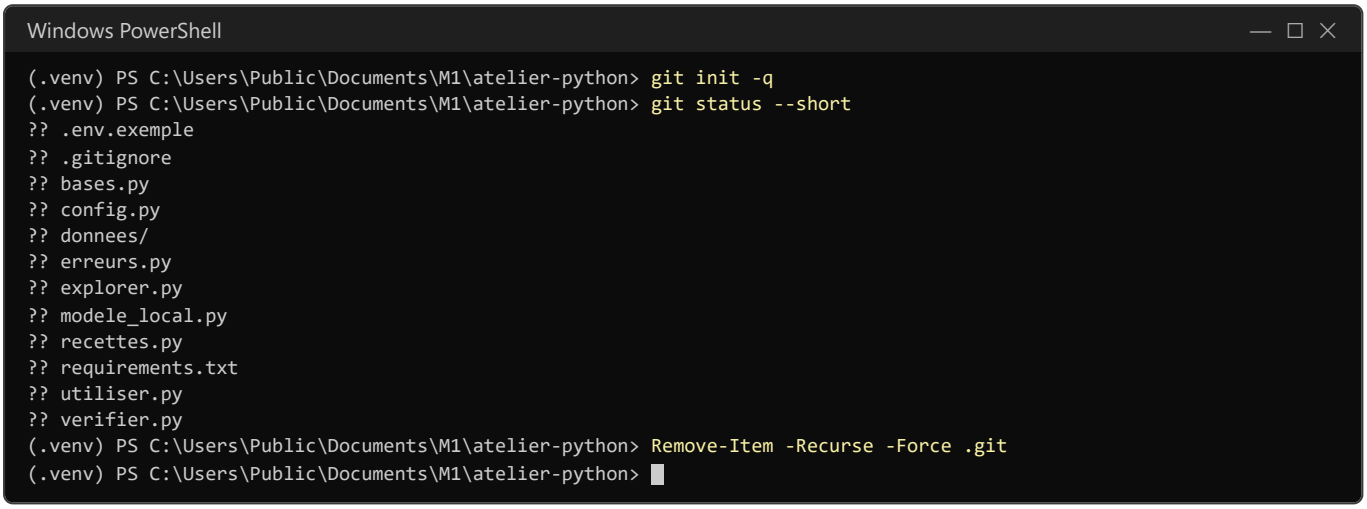
C'est ainsi que les serveurs de production reçoivent leur configuration : pas de fichier `.env` sur le serveur, des variables d'environnement posées par l'hébergeur.

8.6 Vérifier que le secret ne partirait pas

Si Git est installé sur votre machine, créez un dépôt de test, demandez-lui ce qu'il verrait, puis supprimez ce dépôt de test. Si `git` est une commande inconnue, lisez simplement la capture.

Windows (PowerShell)	macOS (Terminal)

`git init -q`
`git status --short`
`Remove-Item -Recurse -Force .git`
	`git init -q`
	`git status --short`
	`rm -rf .git`



`??` marque un fichier que Git pourrait ajouter. `.env.exemple` y est, `.env` n'y est pas, pas plus que `.venv` ni `__pycache__` : le `.gitignore` fait son travail.

À vous (E9)

Ajoutez au `.env` une variable `MODELE_TEMPERATURE=0.2`, et à `config.py` une constante `TEMPERATURE` qui la lit, avec `0.7` par défaut. Attention : `os.getenv` rend toujours un texte. Affichez la valeur et son type : il doit être `float`.

Partie 9. Lancer le modèle local et l'appeler en HTTP

Une API de modèle de langage est un **serveur web**. On lui envoie une requête HTTP qui contient la question en JSON, il renvoie la réponse en JSON. Le fichier `modele_local.py` est un tel serveur, qui imite l'API d'OpenAI. Vous ne le modifiez jamais : vous le lancez, et vos programmes lui parlent.

9.1 Deux terminaux

Un serveur tourne en continu et occupe son terminal. Il vous en faut donc deux, **chacun ouvert dans `atelier-python`, chacun avec l'environnement activé** :

- **Terminal 1**, pour le modèle. Vous le lancez une fois, puis vous n'y touchez plus.
- **Terminal 2**, pour vos programmes. C'est celui où vous tapez toutes les commandes à partir d'ici.

Windows (PowerShell)	macOS (Terminal)

Dans VS Code, le bouton **+** du panneau Terminal ouvre un second terminal dans le dossier du projet. Activez-y l'environnement :
`.venv\Scripts\Activate.ps1`
	Dans VS Code, le bouton **+** du panneau Terminal. Dans l'application Terminal, Cmd+T ouvre un onglet, puis :
	`cd` vers le dossier `atelier-python`
	`source .venv/bin/activate`

9.2 Lancer le modèle dans le terminal 1

Windows (PowerShell)	macOS (Terminal)
`python modele_local.py`	`python modele_local.py`

Terminal 1 : le modèle local (ne pas fermer)

```
(.venv) PS C:\Users\Public\Documents\M1\atelier-python> python modele_local.py
Modèle local prêt sur http://127.0.0.1:8000/v1  (Ctrl+C pour arrêter)
```

Le serveur écoute à l'adresse `http://127.0.0.1:8000/v1`. `127.0.0.1` désigne votre propre machine, `8000` est le **port**, une sorte de numéro de porte. L'invite ne revient pas : c'est normal, le serveur attend les requêtes. Pour l'arrêter plus tard, Ctrl+C dans ce terminal, sous Windows comme sous macOS.

Sous Windows, le pare-feu peut demander s'il faut autoriser Python sur le réseau : vous pouvez refuser, le serveur n'écoute que votre machine.

9.3 Si le port est déjà pris

Si vous relancez le modèle alors qu'il tourne déjà, par exemple dans un terminal oublié, le second lancement s'arrête aussitôt :

Terminal 2 : vos programmes

```
(.venv) PS C:\Users\Public\Documents\M1\atelier-python> python modele_local.py
Le port 8000 est déjà pris : un modèle local tourne sans doute déjà dans un autre terminal.
(.venv) PS C:\Users\Public\Documents\M1\atelier-python>
```

Un port ne peut servir qu'à un programme à la fois. Retrouvez le terminal où le modèle tourne déjà, ou fermez-le et relancez.

9.4 Une première requête : la liste des modèles

La bibliothèque `requests` envoie une requête HTTP en une ligne. `requests.get` demande une ressource. La clé voyage dans un **en-tête** HTTP, `Authorization`, sous la forme `Bearer` suivi de la clé : c'est la convention de toutes les API de modèles. `timeout=10` abandonne après dix secondes plutôt que d'attendre sans fin. Dans le terminal 2, créez `appel_requests.py` :

appel_requests.py

```
1  # appel_requests.py : parler au modèle avec de simples requêtes HTTP
2  import requests
3  import config
4
5  r = requests.get(f"{config.URL}/models",
6                   headers={"Authorization": f"Bearer {config.CLE}"},
7                   timeout=10)
8  print("Code HTTP :", r.status_code)
9  print("Corps :", r.json())
```

Windows (PowerShell), terminal 2	macOS (Terminal), terminal 2
`python appel_requests.py`	`python appel_requests.py`

Terminal 2 : vos programmes

```
(.venv) PS C:\Users\Public\Documents\M1\atelier-python> python appel_requests.py
Code HTTP : 200
Corps : {'object': 'list', 'data': [{'id': 'modele-local', 'object': 'model', 'created': 1767225600, 'owned_by': 'ate
lier'}]}
(.venv) PS C:\Users\Public\Documents\M1\atelier-python>
```

Le **code HTTP** 200 signifie « réussi ». `r.json()` a converti le corps de la réponse en dictionnaire Python : la liste des modèles disponibles, ici un seul.

9.5 Sans la clé

Retirez l'en-tête pour voir ce que répond un fournisseur à un appel anonyme :

appel_requests.py

```
   (lignes 1 à 2 inchangées)
3  import config
4
   r = requests.get(f"{config.URL}/models",
                    headers={"Authorization": f"Bearer {config.CLE}"},
                    timeout=10)
5  r = requests.get(f"{config.URL}/models", timeout=10)
6  print("Code HTTP :", r.status_code)
7  print("Corps :", r.json())
```

Terminal 2 : vos programmes

```
(.venv) PS C:\Users\Public\Documents\M1\atelier-python> python appel_requests.py
Code HTTP : 401
Corps : {'error': {'message': "Clé absente ou invalide. Envoyez l'en-tete Authorization: Bearer sk-...", 'type': 'aut
hentication_error', 'code': 401}}
(.venv) PS C:\Users\Public\Documents\M1\atelier-python>
```

Le code 401 signifie « non authentifié ». Le serveur répond quand même, avec un message d'erreur en JSON. Retenez les codes les plus fréquents : 200 réussi, 400 requête mal formée, 401 clé absente ou fausse, 429 trop de requêtes, 500 erreur du serveur.

9.6 Poser une question

On pose une question par un `POST` sur la route `/chat/completions`. Le corps, passé avec `json=`, contient le nom du modèle, la liste des **messages** (chacun avec un rôle et un contenu) et la **température**, qui règle la part de hasard de la réponse : 0 donne toujours la même réponse. `raise_for_status()` lève une erreur si le code HTTP n'est pas un succès. Remplacez le contenu du fichier :

appel_requests.py

```python
# appel_requests.py : parler au modèle avec de simples requêtes HTTP
import requests
import config

r = requests.get(f"{config.URL}/models", timeout=10)
print("Code HTTP :", r.status_code)
print("Corps :", r.json())

def demander(question, temperature=0.7):
    """Envoie une question au modèle et rend le JSON de sa réponse."""
    r = requests.post(
        f"{config.URL}/chat/completions",
        headers={"Authorization": f"Bearer {config.CLE}"},
        json={
            "model": config.NOM,
            "messages": [{"role": "user", "content": question}],
            "temperature": temperature,
        },
        timeout=10,
    )
    r.raise_for_status()
    return r.json()


if __name__ == "__main__":
    donnees = demander("Par quoi remplacer le beurre dans une tarte ?")
    print("Réponse :", donnees["choices"][0]["message"]["content"])
    print("Jetons :", donnees["usage"])
```

Terminal 2 : vos programmes

```
(.venv) PS C:\Users\Public\Documents\M1\atelier-python> python appel_requests.py
Réponse : La plupart des ingrédients se remplacent : dites-moi lequel manque et dans quelle recette.
Jetons : {'prompt_tokens': 11, 'completion_tokens': 22, 'total_tokens': 33}
(.venv) PS C:\Users\Public\Documents\M1\atelier-python>
```

La réponse du modèle est rangée profond dans le JSON : `choices[0]["message"]["content"]`, le contenu du message du premier choix. `usage` compte les **jetons** (tokens), les morceaux de mots que le modèle lit et écrit : 11 pour la question, 22 pour la réponse. Les fournisseurs facturent au jeton : ces chiffres sont votre facture.

9.7 Quand le modèle ne répond pas

Arrêtez le modèle dans le terminal 1 avec Ctrl+C, puis relancez `appel_requests.py` dans le terminal 2 :

Terminal 2 : vos programmes

```
(.venv) PS C:\Users\Public\Documents\M1\atelier-python> python appel_requests.py
Traceback (most recent call last):
  File "C:\Users\Public\Documents\M1\atelier-python\.venv\Lib\site-packages\urllib3\connection.py", line 239, in _new
_conn
    sock = connection.create_connection(
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
[ 60 lignes coupées ici pour tenir sur la page ]
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\Public\Documents\M1\atelier-python\.venv\Lib\site-packages\requests\sessions.py", line 784, in send
    r = adapter.send(request, **kwargs)
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\Public\Documents\M1\atelier-python\.venv\Lib\site-packages\requests\adapters.py", line 729, in send
    raise ConnectionError(e, request=request)
requests.exceptions.ConnectionError: HTTPConnectionPool(host='127.0.0.1', port=8000): Max retries exceeded with url:
/v1/chat/completions (Caused by NewConnectionError("HTTPConnection(host='127.0.0.1', port=8000): Failed to establish
a new connection: [WinError 10061] Aucune connexion n’a pu être établie car l’ordinateur cible l’a expressément refus
ée"))
(.venv) PS C:\Users\Public\Documents\M1\atelier-python>
```

Une longue trace, dont seule la fin compte : `requests.exceptions.ConnectionError`, avec pour cause `WinError 10061` sous Windows (`Connection refused` sous macOS). Personne n'écoute au port 8000. Un utilisateur ne doit jamais voir cela. On rattrape les deux cas prévisibles, serveur absent et requête refusée, et on met le cas normal dans un bloc `else`, qui ne s'exécute que si aucune erreur n'est survenue :

appel_requests.py

```
    (lignes 1 à 20 inchangées)
21
22  if __name__ == "__main__":
        ~~donnees = demander("Par quoi remplacer le beurre dans une tarte ?")~~
        ~~print("Réponse :", donnees["choices"][0]["message"]["content"])~~
        ~~print("Jetons :", donnees["usage"])~~
23      try:
24          donnees = demander("Par quoi remplacer le beurre dans une tarte ?")
25      except requests.ConnectionError:
26          print("Le modèle ne répond pas : est-il lancé dans l'autre terminal ?")
27      except requests.HTTPError as err:
28          print("Le modèle a refusé la requête :", err.response.status_code, err.response.json()["error"]
    ["message"])
29      else:
30          print("Réponse :", donnees["choices"][0]["message"]["content"])
31          print("Jetons :", donnees["usage"])
```

Serveur toujours arrêté, relancez :

```
Terminal 2 : vos programmes
(.venv) PS C:\Users\Public\Documents\M1\atelier-python> python appel_requests.py
Le modèle ne répond pas : est-il lancé dans l'autre terminal ?
(.venv) PS C:\Users\Public\Documents\M1\atelier-python>
```

Un message clair, qui dit quoi faire. Relancez maintenant le modèle dans le terminal 1 (flèche du haut, Entrée). Pour provoquer l'autre cas, remplacez un instant `MODELE_CLE=sk-atelier-local` par `MODELE_CLE=mauvaise` dans `.env`, relancez :

```
Terminal 2 : vos programmes
(.venv) PS C:\Users\Public\Documents\M1\atelier-python> python appel_requests.py
Le modèle a refusé la requête : 401 Clé absente ou invalide. Envoyez l'en-tete Authorization: Bearer sk-...
(.venv) PS C:\Users\Public\Documents\M1\atelier-python>
```

`err.response` donne accès à la réponse du serveur, donc au code 401 et au message. **Remettez la bonne clé dans `.env`**, puis vérifiez que tout remarche :

```
Terminal 2 : vos programmes
(.venv) PS C:\Users\Public\Documents\M1\atelier-python> python appel_requests.py
Réponse : La plupart des ingrédients se remplacent : dites-moi lequel manque et dans quelle recette.
Jetons : {'prompt_tokens': 11, 'completion_tokens': 22, 'total_tokens': 33}
(.venv) PS C:\Users\Public\Documents\M1\atelier-python>
```

À vous (E10)

Dans le bloc `__main__`, ajoutez un appel à `demander("Est-ce que le gratin se congèle ?", temperature=0)` et affichez la réponse et le détail des jetons. Combien de jetons la question a-t-elle coûté, et combien la réponse ?

Partie 10. Le même appel avec le SDK openai

Un **SDK** est une bibliothèque officielle qui emballe les requêtes HTTP d'une API dans des fonctions Python. Le SDK `openai` construit la requête, ajoute l'en-tête de clé, vérifie le code HTTP et convertit la réponse en objets. Il parle à toute API qui suit le format d'OpenAI, y compris votre modèle local : il suffit de lui donner une autre adresse avec `base_url`. Le modèle doit tourner dans le terminal 1.

Le SDK est gratuit et libre : `openai` est publié sous licence Apache 2.0, comme `requests` ; `pydantic` et `pytest` sont sous licence MIT, `python-dotenv` sous licence BSD. Ce qui se paie, c'est le service d'OpenAI, au jeton, et seulement si vous lui donnez une vraie clé. Ici le SDK parle à votre modèle local : aucun compte, aucun paiement. Le même code parle aussi à un modèle libre qui tourne sur votre machine, par exemple avec Ollama, en changeant seulement `base_url` et le nom du modèle.

10.1 Un appel avec un message système

Le message de rôle `system` donne au modèle ses consignes générales ; le message `user` porte la question. Créez `appel_sdk.py` :

appel_sdk.py

```python
# appel_sdk.py : le même appel avec le SDK officiel openai
from openai import OpenAI
import config

client = OpenAI(base_url=config.URL, api_key=config.CLE)

reponse = client.chat.completions.create(
    model=config.NOM,
    messages=[
        {"role": "system", "content": "Tu es l'assistant d'un site de recettes."},
        {"role": "user", "content": "Combien de jours garder une soupe au frigo ?"},
    ],
    temperature=0,
)
print("Type de l'objet :", type(reponse).__name__)
print("Réponse :", reponse.choices[0].message.content)
print("Jetons :", reponse.usage.prompt_tokens, "+", reponse.usage.completion_tokens,
      "=", reponse.usage.total_tokens)
```

Windows (PowerShell), terminal 2	macOS (Terminal), terminal 2
`python appel_sdk.py`	`python appel_sdk.py`

Terminal 2 : vos programmes

```
(.venv) PS C:\Users\Public\Documents\M1\atelier-python> python appel_sdk.py
Type de l'objet : ChatCompletion
Réponse : La plupart des plats cuits se gardent trois jours au frais, dans une boîte fermée.
Jetons : 21 + 20 = 41
(.venv) PS C:\Users\Public\Documents\M1\atelier-python>
```

La réponse n'est plus un dictionnaire mais un objet `ChatCompletion` : on lit ses champs avec des points, `reponse.choices[0].message.content`, et VS Code les propose pendant la frappe. Le chemin est le même que dans le JSON de la partie 9.

10.2 Tenir une conversation

Un modèle de langage n'a **aucune mémoire** d'un appel à l'autre. Pour qu'il se souvienne de la question précédente, on lui renvoie à chaque appel tout l'historique : les questions, et ses propres réponses avec le rôle `assistant`. Ajoutez à la fin du fichier :

appel_sdk.py

```
(lignes 1 à 16 inchangées)
print("Jetons :", reponse.usage.prompt_tokens, "+", reponse.usage.completion_tokens,
      "=", reponse.usage.total_tokens)

historique = [{"role": "system", "content": "Tu es l'assistant d'un site de recettes."}]
for question in ["Je n'ai plus de crème, je mets quoi ?", "Et pour la conserver ?"]:
    historique.append({"role": "user", "content": question})
    rep = client.chat.completions.create(model=config.NOM, messages=historique)
    texte = rep.choices[0].message.content
    historique.append({"role": "assistant", "content": texte})
    print(f"Vous : {question}")
    print(f"Modèle : {texte}")
print("Messages envoyés au dernier appel :", len(historique) - 1)
```

Terminal 2 : vos programmes

```
(.venv) PS C:\Users\Public\Documents\M1\atelier-python> python appel_sdk.py
Type de l'objet : ChatCompletion
Réponse : La plupart des plats cuits se gardent trois jours au frais, dans une boîte fermée.
Jetons : 21 + 20 = 41
Vous : Je n'ai plus de crème, je mets quoi ?
Modèle : La plupart des ingrédients se remplacent : dites-moi lequel manque et dans quelle recette.
Vous : Et pour la conserver ?
Modèle : La plupart des plats cuits se gardent trois jours au frais, dans une boîte fermée.
Messages envoyés au dernier appel : 4
(.venv) PS C:\Users\Public\Documents\M1\atelier-python>
```

La seconde question, « Et pour la conserver ? », ne veut rien dire seule : le modèle comprend qu'il s'agit de la crème parce que l'échange précédent est dans la liste. Au dernier appel, quatre messages sont partis : le système, la première question, la première réponse et la seconde question. **L'historique grossit à chaque tour, et chaque jeton est facturé à chaque appel** : c'est pourquoi les applications réelles tronquent ou résument les vieilles conversations.

10.3 L'erreur de l'environnement oublié

Désactivez l'environnement, lancez le script, puis réactivez :

Windows (PowerShell), terminal 2	macOS (Terminal), terminal 2

`deactivate`
`python appel_sdk.py`
`.venv\Scripts\Activate.ps1`
	`deactivate`
	`python3 appel_sdk.py`
	`source .venv/bin/activate`

Terminal 2 : vos programmes

```
(.venv) PS C:\Users\Public\Documents\M1\atelier-python> deactivate
PS C:\Users\Public\Documents\M1\atelier-python> python appel_sdk.py
Traceback (most recent call last):
  File "C:\Users\Public\Documents\M1\atelier-python\appel_sdk.py", line 2, in <module>
    from openai import OpenAI
ModuleNotFoundError: No module named 'openai'
PS C:\Users\Public\Documents\M1\atelier-python> .venv\Scripts\Activate.ps1
(.venv) PS C:\Users\Public\Documents\M1\atelier-python>
```

`ModuleNotFoundError: No module named 'openai'` ne veut presque jamais dire qu'il faut réinstaller : cela veut dire que le terminal n'utilise pas l'environnement où la bibliothèque est installée. Regardez l'invite avant de lancer `pip install` .

À vous (E11)

Ajoutez une troisième question à la liste : « Et si un invité est allergique aux arachides ? ». Relancez. Que répond le modèle, et combien de messages sont partis au dernier appel ? Expliquez ce nombre.

Partie 11. Ne croire que les réponses valides

Dès qu'un programme utilise la réponse d'un modèle, par exemple pour aiguiller une question vers le bon service, il lui faut une réponse structurée : un JSON avec des champs connus. Or un modèle produit du texte, et ce texte n'est pas toujours conforme. **Pydantic** permet de décrire la forme attendue une fois, puis de vérifier chaque réponse contre cette description.

11.1 Décrire la réponse attendue

Une **classe** qui hérite de `BaseModel` déclare ses champs avec leur type. `Literal[...]` restreint une valeur à une liste fermée. `Field(ge=0, le=1)` impose une valeur entre 0 et 1 (*greater or equal*, *less or equal*). Créez `schemas.py` :

schemas.py

```python
"""Le format attendu d'une réponse de classement."""
from typing import Literal
from pydantic import BaseModel, Field, ValidationError

Intention = Literal["recette_frigo", "substitution", "cuisson_technique",
                    "conservation", "allergie_sante", "compte_abonnement", "autre"]


class Classement(BaseModel):
    intention: Intention
    confiance: float = Field(ge=0, le=1)
    reponse: str


if __name__ == "__main__":
    bon = Classement(intention="conservation", confiance=0.88, reponse="Trois jours au frais.")
    print("Valide :", bon)
    print("Confiance :", bon.confiance, type(bon.confiance).__name__)
    try:
        Classement(intention="dessert", confiance=1.4, reponse="?")
    except ValidationError as err:
        print(err)
```

Windows (PowerShell), terminal 2	macOS (Terminal), terminal 2
`python schemas.py`	`python schemas.py`

Terminal 2 : vos programmes

```
(.venv) PS C:\Users\Public\Documents\M1\atelier-python> python schemas.py
Valide : intention='conservation' confiance=0.88 reponse='Trois jours au frais.'
Confiance : 0.88 float
2 validation errors for Classement
intention
  Input should be 'recette_frigo', 'substitution', 'cuisson_technique', 'conservation', 'allergie_sante', 'compte_abo
nnement' or 'autre' [type=literal_error, input_value='dessert', input_type=str]
    For further information visit https://errors.pydantic.dev/2.13/v/literal_error
confiance
  Input should be less than or equal to 1 [type=less_than_equal, input_value=1.4, input_type=float]
    For further information visit https://errors.pydantic.dev/2.13/v/less_than_equal
(.venv) PS C:\Users\Public\Documents\M1\atelier-python>
```

Le premier objet est valide, et `confiance` est bien un `float`. Le second est refusé avec deux erreurs précises : « dessert » n'est pas une intention prévue, et 1,4 dépasse 1. Pydantic ne s'arrête pas à la première erreur, il les liste toutes.

11.2 Demander un classement au modèle

On pose douze fois la même question au modèle, en exigeant du JSON, et on valide chaque réponse avec `Classement.model_validate_json`, qui lit le texte JSON et le vérifie en une seule opération. Créez `classer.py` :

classer.py

```python
# classer.py : demander un classement au modèle et ne croire que ce qui est valide
from openai import OpenAI
from pydantic import ValidationError
import config
from schemas import Classement

CONSIGNE = ("Tu classes la question d'un utilisateur d'un site de recettes. "
            "Réponds uniquement en JSON avec les clés intention, confiance et reponse.")

client = OpenAI(base_url=config.URL, api_key=config.CLE)


def demander(question):
    """Rend le texte brut renvoyé par le modèle."""
    reponse = client.chat.completions.create(
        model=config.NOM,
        temperature=0,
        messages=[{"role": "system", "content": CONSIGNE},
                  {"role": "user", "content": question}],
    )
    return reponse.choices[0].message.content


if __name__ == "__main__":
    rejets = 0
    for n in range(12):
        brut = demander("Est-ce que le gratin se congèle ?")
        try:
            c = Classement.model_validate_json(brut)
            print(n + 1, "valide  ", c.intention, c.confiance)
        except ValidationError:
            rejets += 1
            print(n + 1, "REJETÉ  ", brut)
    print("Rejetés :", rejets, "sur 12")
```

Windows (PowerShell), terminal 2	macOS (Terminal), terminal 2
`python classer.py`	`python classer.py`

Terminal 2 : vos programmes

```
(.venv) PS C:\Users\Public\Documents\M1\atelier-python> python classer.py
1 REJETÉ   ```json
{"intention": "conservation", "confiance": 0.72, "reponse": "La plupart des plats cuits se gardent trois jours au fra
is, dans une boîte fermée."}
```
2 valide   conservation 0.72
3 valide   conservation 0.72
4 REJETÉ   {"intention": "conservation", "confiance": 0.72, "reponse": "La plupart des plats cuits se gardent trois j
ours au frais, dans une boîte fermée.",}
5 valide   conservation 0.72
6 valide   conservation 0.72
7 valide   conservation 0.72
8 valide   conservation 0.72
9 REJETÉ   ```json
{"intention": "conservation", "confiance": 0.72, "reponse": "La plupart des plats cuits se gardent trois jours au fra
is, dans une boîte fermée."}
```
10 REJETÉ   Voici le classement demandé : {"intention": "conservation", "confiance": 0.72, "reponse": "La plupart des
plats cuits se gardent trois jours au frais, dans une boîte fermée."}
11 valide   conservation 0.72
12 valide   conservation 0.72
Rejetés : 4 sur 12
(.venv) PS C:\Users\Public\Documents\M1\atelier-python>
```

Quatre réponses sur douze sont rejetées, alors que le contenu est juste à chaque fois. Lisez-les : le JSON est entouré de balises de code `` ```json ``, ou suivi d'une virgule en trop, ou précédé d'une phrase de politesse. C'est le comportement réel des modèles, et c'est pourquoi on ne passe jamais une réponse brute à la suite du programme.

11.3 Nettoyer les défauts connus

Deux des trois défauts sont des défauts de forme, qu'on peut corriger sans risque. Ajoutez une fonction `nettoyer` sous `demander`, et appelez-la avant la validation :

classer.py

```
    (lignes 1 à 21 inchangées)
22
23
24  def nettoyer(texte):
25      """Retire les défauts de forme connus du modèle."""
26      texte = texte.strip()
27      if texte.startswith("```"):
28          texte = texte.strip("`").removeprefix("json").strip()
29      return texte.replace(",}", "}")
30
31
32  if __name__ == "__main__":
33      rejets = 0
    (ligne 34 inchangée)
35          brut = demander("Est-ce que le gratin se congèle ?")
36          try:
            ~~            c = Classement.model_validate_json(brut)~~
37              c = Classement.model_validate_json(nettoyer(brut))
38              print(n + 1, "valide  ", c.intention, c.confiance)
39          except ValidationError:
    (lignes 40 à 42 inchangées)
```

Terminal 2 : vos programmes

```
(.venv) PS C:\Users\Public\Documents\M1\atelier-python> python classer.py
1 valide   conservation 0.72
2 valide   conservation 0.72
3 valide   conservation 0.72
4 valide   conservation 0.72
5 valide   conservation 0.72
6 valide   conservation 0.72
7 valide   conservation 0.72
8 valide   conservation 0.72
9 valide   conservation 0.72
10 valide   conservation 0.72
11 valide   conservation 0.72
12 valide   conservation 0.72
Rejetés : 0 sur 12
(.venv) PS C:\Users\Public\Documents\M1\atelier-python>
```

Zéro rejet. Le nettoyage ne corrige que des défauts **connus et observés** : il ne devine rien. La phrase de politesse, elle, n'est pas retirée ; si elle revient, la validation la rejettera, et c'est le rôle de l'étape suivante d'en tirer quelque chose.

11.4 Relancer, journaliser, abandonner proprement

On réécrit `classer.py` autour d'une fonction `classer(question)` qui **relance** jusqu'à trois fois si la réponse est invalide, et rend `None` en cas d'abandon. Les rejets ne sont plus affichés avec `print` mais notés avec `logging`, le journal standard de Python, qui classe les messages par gravité (INFO, WARNING, ERROR) et qu'on peut rendre plus ou moins bavard sans toucher au code. Remplacez tout le fichier ; voici la première moitié :

classer.py

```
# classer.py : demander un classement au modèle et ne croire que ce qui est valide
import logging
from openai import OpenAI
from pydantic import ValidationError
import config
from schemas import Classement

CONSIGNE = ("Tu classes la question d'un utilisateur d'un site de recettes. "
            "Réponds uniquement en JSON avec les clés intention, confiance et reponse.")

client = OpenAI(base_url=config.URL, api_key=config.CLE)
_
_
def demander(question):
    """Rend le texte brut renvoyé par le modèle."""
    reponse = client.chat.completions.create(
        model=config.NOM,
        temperature=0,
        messages=[{"role": "system", "content": CONSIGNE},
                  {"role": "user", "content": question}],
    )
    return reponse.choices[0].message.content
journal = logging.getLogger(__name__)
CLIENT = OpenAI(base_url=config.URL, api_key=config.CLE)


def nettoyer(texte):
    """Retire les défauts de forme connus du modèle."""
    texte = texte.strip()
    if texte.startswith("```"):
        texte = texte.strip("`").removeprefix("json").strip()
    return texte.replace(",}", "}")


```

- `CLIENT` est créé une fois, au chargement du module. Le paramètre `client=None` permet d'en passer un autre : c'est ce qui rendra la fonction testable sans modèle, à la partie 12.
- `client = client or CLIENT` : si aucun client n'est fourni, on prend celui par défaut.

Seconde moitié, à la suite :

classer.py

```
    (lignes 1 à 22 inchangées)
def classer(question, essais=3, client=None):
    """Rend un Classement valide, ou None après plusieurs essais ratés."""
    client = client or CLIENT
    for essai in range(1, essais + 1):
        reponse = client.chat.completions.create(
            model=config.NOM,
            temperature=0,
            messages=[{"role": "system", "content": CONSIGNE},
                      {"role": "user", "content": question}],
        )
        brut = reponse.choices[0].message.content
        try:
            return Classement.model_validate_json(nettoyer(brut))
        except ValidationError as err:
            journal.info("essai %d rejeté (%d erreur) : %s", essai, err.error_count(), brut)
    journal.warning("abandon après %d essais : %s", essais, question)
    return None


if __name__ == "__main__":
    rejets = 0
    for n in range(12):
        brut = demander("Est-ce que le gratin se congèle ?")
        try:
            c = Classement.model_validate_json(nettoyer(brut))
            print(n + 1, "valide  ", c.intention, c.confiance)
        except ValidationError:
            rejets += 1
            print(n + 1, "REJETÉ  ", brut)
    print("Rejetés :", rejets, "sur 12")
    logging.basicConfig(level=logging.WARNING, format="%(levelname)s %(message)s")
    journal.setLevel(logging.INFO)
    for question in ["Est-ce que le gratin se congèle ?",
                     "On m'a prélevé deux fois mon abonnement",
                     "Je suis allergique aux arachides, je peux faire ce gâteau ?",
                     "Comment réchauffer un reste de risotto ?",
                     "Vous livrez à Lyon ?"]:
        c = classer(question)
        print(f"{question}\n  {c.intention} ({c.confiance}) : {c.reponse}")
```

- Le `return` est à l'intérieur du `try` : dès qu'une réponse est valide, la fonction s'arrête et la rend. Les essais suivants n'ont pas lieu.
- `journal.info("essai %d rejeté ...", essai, ...)` : avec `logging`, on passe les valeurs à part, et non dans une f-string. Le texte n'est fabriqué que si le message est effectivement affiché.

Terminal 2 : vos programmes

```
(.venv) PS C:\Users\Public\Documents\M1\atelier-python> python classer.py
Est-ce que le gratin se congèle ?
  conservation (0.72) : La plupart des plats cuits se gardent trois jours au frais, dans une boîte fermée.
INFO essai 1 rejeté (1 erreur) : Voici le classement demandé : {"intention": "compte_abonnement", "confiance": 0.93,
"reponse": "Une question de compte ou de paiement est traitée par l'équipe du site."}
INFO essai 2 rejeté (1 erreur) : Voici le classement demandé : {"intention": "compte_abonnement", "confiance": 0.93,
"reponse": "Une question de compte ou de paiement est traitée par l'équipe du site."}
On m'a prélevé deux fois mon abonnement
  compte_abonnement (0.93) : Une question de compte ou de paiement est traitée par l'équipe du site.
Je suis allergique aux arachides, je peux faire ce gâteau ?
  allergie_sante (0.88) : Je ne peux pas garantir qu'une recette est sans risque pour une allergie : un humain vous r
épond.
Comment réchauffer un reste de risotto ?
  conservation (0.72) : La plupart des plats cuits se gardent trois jours au frais, dans une boîte fermée.
Vous livrez à Lyon ?
  autre (0.28) : Je transmets votre question à l'équipe du site.
(.venv) PS C:\Users\Public\Documents\M1\atelier-python>
```

Cinq questions, cinq classements. Les lignes INFO montrent les essais rejetés puis rattrapés par une relance : sans elles, vous ne sauriez pas que le modèle s'est trompé de forme. La dernière question, hors sujet, est classée `autre` avec une confiance basse : le programme pourra la traiter à part.

À vous (E12)

Dans l'interpréteur, tapez `from schemas import Classement`, puis `Classement(intention="autre", confiance=0.5)`. Quelle erreur obtenez-vous, et que dit-elle ? Qu'en déduisez-vous sur une réponse de modèle qui oublierait un champ ?

Partie 12. Écrire des tests avec pytest

Un **test** est une petite fonction qui appelle votre code avec des données connues et vérifie le résultat avec `assert`. Si l'affirmation est fausse, le test échoue. **pytest** trouve et lance tous les tests d'un coup : il cherche les fichiers dont le nom commence par `test_`, et dans ces fichiers les fonctions qui commencent par `test_`.

12.1 Le dossier des tests

Windows (PowerShell), terminal 2	macOS (Terminal), terminal 2
`New-Item -ItemType Directory tests`	`mkdir tests`

Créez dans ce dossier le fichier `test_recettes.py`. Les tests utilisent une petite liste `EXEMPLE` écrite à la main, dont on connaît le bon résultat par cœur. `pytest.raises(ValueError)` vérifie qu'une erreur est bien levée.

tests\test_recettes.py

```python
# tests/test_recettes.py : vérifier le module recettes sans rien afficher
import pytest
from recettes import charger, filtrer_par_temps, chercher_ingredient

EXEMPLE = [
    {"titre": "A", "temps_min": 10, "ingredients": ["Oeufs", "lait"]},
    {"titre": "B", "temps_min": 40, "ingredients": ["farine"]},
]


def test_le_fichier_contient_15_recettes():
    assert len(charger()) == 15


def test_filtrer_garde_les_rapides():
    assert [r["titre"] for r in filtrer_par_temps(EXEMPLE, 20)] == ["A"]


def test_filtrer_inclut_la_limite():
    assert len(filtrer_par_temps(EXEMPLE, 40)) == 2


def test_filtrer_refuse_un_temps_negatif():
    with pytest.raises(ValueError):
        filtrer_par_temps(EXEMPLE, -1)


def test_chercher_ignore_les_majuscules():
    assert [r["titre"] for r in chercher_ingredient(EXEMPLE, "oeufs")] == ["A"]
```

Remarquez le premier ingrédient de la recette A : `"Oeufs"`, avec une majuscule. Le dernier test vérifie qu'une recherche en minuscules le trouve quand même.

12.2 Lancer pytest, et la bonne façon de le faire

Windows (PowerShell), terminal 2	macOS (Terminal), terminal 2
`pytest -q`	`pytest -q`

Terminal 2 : vos programmes

```
(.venv) PS C:\Users\Public\Documents\M1\atelier-python> pytest -q
================================== ERRORS ==================================
____________________ ERROR collecting tests/test_recettes.py ____________________
ImportError while importing test module 'C:\Users\Public\Documents\M1\atelier-python\tests\test_recettes.py'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
C:\Program Files\Python312\Lib\importlib\__init__.py:90: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
tests\test_recettes.py:3: in <module>
    from recettes import charger, filtrer_par_temps, chercher_ingredient
E   ModuleNotFoundError: No module named 'recettes'
=========================== short test summary info ============================
ERROR tests/test_recettes.py
!!!!!!!!!!!!!!!!!!!! Interrupted: 1 error during collection !!!!!!!!!!!!!!!!!!!!
1 error in 0.07s
(.venv) PS C:\Users\Public\Documents\M1\atelier-python>
```

Le test ne trouve pas le module `recettes`, pourtant présent. Lancé par la commande `pytest`, l'outil ne met pas le dossier courant dans les chemins où Python cherche les modules. Lancé par `python -m pytest`, si : c'est la même logique que `python -m pip`. Prenez l'habitude de cette forme.

Windows (PowerShell), terminal 2	macOS (Terminal), terminal 2
`python -m pytest -q`	`python -m pytest -q`

Terminal 2 : vos programmes

```
(.venv) PS C:\Users\Public\Documents\M1\atelier-python> python -m pytest -q
....F                                                                    [100%]
================================== FAILURES ===================================
_____________________ test_chercher_ignore_les_majuscules _____________________

    def test_chercher_ignore_les_majuscules():
>       assert [r["titre"] for r in chercher_ingredient(EXEMPLE, "oeufs")] == ["A"]
E       AssertionError: assert [] == ['A']
E
E         Right contains one more item: 'A'
E         Use -v to get more diff

tests\test_recettes.py:29: AssertionError
=========================== short test summary info ============================
FAILED tests/test_recettes.py::test_chercher_ignore_les_majuscules - Assertio...
1 failed, 4 passed in 0.03s
(.venv) PS C:\Users\Public\Documents\M1\atelier-python>
```

Quatre tests passent, un échoue, et c'est une vraie trouvaille. pytest montre la ligne fautive, précédée de `>`, et les deux valeurs comparées : la fonction a rendu une liste vide au lieu de `['A']`. `chercher_ingredient` met le mot cherché en minuscules, mais compare avec l'ingrédient tel quel, « Oeufs » : les majuscules des données ne sont jamais converties.

12.3 Corriger le code, pas le test

Le test décrit le comportement voulu ; c'est le module qui a tort. Dans `recettes.py`, une seule ligne change :

recettes.py

```
(lignes 1 à 24 inchangées)
    for r in recettes:
        for i in r["ingredients"]:
-           if cherche in i:
+           if cherche in i.lower():
                trouvees.append(r)
                break
(lignes 30 à 43 inchangées)
```

Terminal 2 : vos programmes

```
(.venv) PS C:\Users\Public\Documents\M1\atelier-python> python -m pytest -q
.....                                                                    [100%]
5 passed in 0.01s
(.venv) PS C:\Users\Public\Documents\M1\atelier-python>
```

Cinq tests passent. Ce défaut était déjà là à la partie 6, sans que rien ne le montre : les données de l'atelier sont toutes en minuscules. Il aurait surgi le jour où un fichier aurait contenu une majuscule. C'est tout l'intérêt d'un test : il essaie le cas auquel on n'a pas pensé en écrivant la fonction, et il continue de l'essayer à chaque modification.

12.4 Tester sans appeler le modèle

Un test qui appelle un vrai modèle est lent, coûte des jetons, et ne donne pas toujours le même résultat. On remplace donc le client par un **faux client** qui rend des réponses écrites d'avance et compte les appels. C'est possible parce que `classer()` accepte un paramètre `client`. `SimpleNamespace` fabrique un objet dont on choisit les attributs : ici, de quoi imiter `client.chat.completions.create(...)` et `reponse.choices[0].message.content`. Créez `tests/test_classer.py`, première moitié :

tests\test_classer.py

```
# tests/test_classer.py : tester classer() sans appeler le modèle
from types import SimpleNamespace
import classer


class FauxClient:
    """Imite le client openai : rend des textes écrits d'avance et compte les appels."""

    def __init__(self, textes):
        self.textes = list(textes)
        self.appels = 0
        self.chat = SimpleNamespace(completions=SimpleNamespace(create=self.create))

    def create(self, **kwargs):
        self.appels += 1
        message = SimpleNamespace(content=self.textes.pop(0))
        return SimpleNamespace(choices=[SimpleNamespace(message=message)])


BON = '{"intention": "autre", "confiance": 0.5, "reponse": "ok"}'


```

tests\test_classer.py

```
    (lignes 1 à 22 inchangées)
def test_nettoyer_retire_les_balises():
    assert classer.nettoyer("```json\n" + BON + "\n```") == BON


def test_nettoyer_retire_la_virgule_en_trop():
    assert classer.nettoyer('{"a": 1,}') == '{"a": 1}'


def test_une_reponse_valide_suffit():
    faux = FauxClient([BON])
    assert classer.classer("Bonjour", client=faux).intention == "autre"
    assert faux.appels == 1


def test_relance_apres_une_reponse_invalide():
    faux = FauxClient(["pas du JSON", BON])
    assert classer.classer("Bonjour", client=faux).intention == "autre"
    assert faux.appels == 2


def test_abandon_apres_trois_echecs():
    faux = FauxClient(["x", "y", "z"])
    assert classer.classer("Bonjour", client=faux) is None
    assert faux.appels == 3
```

Les trois derniers tests couvrent les trois chemins de `classer()` : réussite d'emblée, après relance, abandon. `-v` affiche chaque test par son nom :

Windows (PowerShell), terminal 2	macOS (Terminal), terminal 2
`python -m pytest -v`	`python -m pytest -v`

Terminal 2 : vos programmes

```
(.venv) PS C:\Users\Public\Documents\M1\atelier-python> python -m pytest -v
============================= test session starts =============================
platform win32 -- Python 3.12.2, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\Public\Documents\M1\atelier-python\.venv\Scri
pts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\Public\Documents\M1\atelier-python
plugins: anyio-4.15.1
collecting ... collected 10 items

tests/test_classer.py::test_nettoyer_retire_les_balises PASSED           [ 10%]
tests/test_classer.py::test_nettoyer_retire_la_virgule_en_trop PASSED    [ 20%]
tests/test_classer.py::test_une_reponse_valide_suffit PASSED             [ 30%]
tests/test_classer.py::test_relance_apres_une_reponse_invalide PASSED    [ 40%]
tests/test_classer.py::test_abandon_apres_trois_echecs PASSED            [ 50%]
tests/test_recettes.py::test_le_fichier_contient_15_recettes PASSED      [ 60%]
tests/test_recettes.py::test_filtrer_garde_les_rapides PASSED            [ 70%]
tests/test_recettes.py::test_filtrer_inclut_la_limite PASSED             [ 80%]
tests/test_recettes.py::test_filtrer_refuse_un_temps_negatif PASSED      [ 90%]
tests/test_recettes.py::test_chercher_ignore_les_majuscules PASSED       [100%]

============================== 10 passed in 0.43s ==============================
(.venv) PS C:\Users\Public\Documents\M1\atelier-python>
```

À vous (E13)

Ajoutez à `tests/test_recettes.py` un test `test_chercher_sans_resultat` : chercher « chocolat » dans `EXEMPLE` rend une liste vide. Combien de tests passent ?

Partie 13. Mesurer la qualité du classement

Les tests vérifient que le code fait ce qu'il doit. Ils ne disent pas si le modèle classe bien les questions. Pour cela, on écrit une **évaluation** : un jeu de questions dont on connaît la bonne réponse, qu'on fait passer au modèle, et un score. C'est le fichier `donnees/questions.csv`, vingt questions et leur intention attendue. Sans évaluation, on ne peut pas dire si un changement de consigne ou de modèle améliore ou dégrade les choses.

13.1 Lire le CSV et calculer l'exactitude

`csv.DictReader` lit un CSV ligne par ligne et rend chaque ligne comme un dictionnaire, dont les clés sont les noms de colonnes. `delimiter=";"` indique le séparateur, `newline=""` est demandé par la documentation du module `csv`. Créez `evaluer.py` :

evaluer.py

```python
# evaluer.py : mesurer le classement sur des questions dont on connaît la bonne réponse
import csv
from classer import classer

with open("donnees/questions.csv", encoding="utf-8", newline="") as f:
    lignes = list(csv.DictReader(f, delimiter=";"))
print(len(lignes), "questions lues")

resultats = []
for ligne in lignes:
    c = classer(ligne["question"])
    obtenue = c.intention if c else "echec"
    resultats.append({
        "question": ligne["question"],
        "attendue": ligne["intention_attendue"],
        "obtenue": obtenue,
        "juste": obtenue == ligne["intention_attendue"],
    })

justes = sum(1 for r in resultats if r["juste"])
print(f"Exactitude : {justes}/{len(resultats)}, soit {justes / len(resultats):.0%}")
```

Windows (PowerShell), terminal 2	macOS (Terminal), terminal 2
`python evaluer.py`	`python evaluer.py`

Terminal 2 : vos programmes

```
(.venv) PS C:\Users\Public\Documents\M1\atelier-python> python evaluer.py
20 questions lues
Exactitude : 17/20, soit 85%
(.venv) PS C:\Users\Public\Documents\M1\atelier-python>
```

17 bonnes réponses sur 20. `sum(1 for r in resultats if r["juste"])` compte les éléments qui vérifient une condition. `:.0%` affiche une fraction en pourcentage sans décimale.

13.2 Regarder les erreurs, et garder une trace

Un score seul ne dit pas quoi corriger. On liste les erreurs, on les compte par intention avec `Counter`, qui compte les occurrences d'une liste en une ligne, et on écrit le détail dans un fichier avec `csv.DictWriter`. L'encodage `utf-8-sig` ajoute une marque en tête de fichier qui permet à Excel d'afficher les accents correctement.

evaluer.py

```
# evaluer.py : mesurer le classement sur des questions dont on connaît la bonne réponse
import csv
from collections import Counter
from classer import classer

(Lignes 6 à 20 inchangées)
justes = sum(1 for r in resultats if r["juste"])
print(f"Exactitude : {justes}/{len(resultats)}, soit {justes / len(resultats):.0%}")

erreurs = Counter(r["attendue"] for r in resultats if not r["juste"])
print("Erreurs par intention attendue :", dict(erreurs))
for r in resultats:
    if not r["juste"]:
        print(f"  {r['question']}\n     attendu {r['attendue']}, obtenu {r['obtenue']}")

with open("resultats.csv", "w", encoding="utf-8-sig", newline="") as f:
    ecrivain = csv.DictWriter(f, fieldnames=["question", "attendue", "obtenue", "juste"], delimiter=";")
    ecrivain.writeheader()
    ecrivain.writerows(resultats)
print("Détail écrit dans resultats.csv")
```

Terminal 2: vos programmes

```
(.venv) PS C:\Users\Public\Documents\M1\atelier-python> python evaluer.py
20 questions lues
Exactitude : 17/20, soit 85%
Erreurs par intention attendue : {'recette_frigo': 1, 'cuisson_technique': 1, 'compte_abonnement': 1}
  Je n'ai pas le temps, une recette en moins de 20 minutes ?
     attendu recette_frigo, obtenu substitution
  Le poulet est encore rose à l'intérieur, c'est normal ?
     attendu cuisson_technique, obtenu autre
  Ma carte a été débitée mais je n'ai rien reçu
     attendu compte_abonnement, obtenu autre
Détail écrit dans resultats.csv
(.venv) PS C:\Users\Public\Documents\M1\atelier-python>
```

Les trois erreurs deviennent lisibles. « Le poulet est encore rose » est une question de cuisson classée `autre` ; « Ma carte a été débitée » est une question d'abonnement classée `autre`. Ce sont les questions formulées sans les mots attendus : c'est là qu'une meilleure consigne ou des exemples dans le prompt feraient gagner des points, et l'évaluation dira si c'est le cas.

Windows (PowerShell), terminal 2	macOS (Terminal), terminal 2
`Get-Content resultats.csv -Encoding UTF8 -TotalCount 5`	`head -n 5 resultats.csv`

Terminal 2 : vos programmes

```
(.venv) PS C:\Users\Public\Documents\M1\atelier-python> Get-Content resultats.csv -Encoding UTF8 -TotalCount 5
question;attendue;obtenue;juste
J'ai des courgettes et des oeufs, je fais quoi ce soir ?;recette_frigo;recette_frigo;True
Il me reste du riz et des champignons, une idée de dîner ?;recette_frigo;recette_frigo;True
Je n'ai pas le temps, une recette en moins de 20 minutes ?;recette_frigo;substitution;False
Je n'ai plus de crème, je peux mettre quoi à la place ?;substitution;substitution;True
(.venv) PS C:\Users\Public\Documents\M1\atelier-python>
```

Le fichier s'ouvre aussi dans Excel ou LibreOffice. Il est listé dans `.gitignore` : il se régénère à chaque exécution.

À vous (E14)

Le jeu de questions est-il équilibré ? Dans un nouveau fichier `compter.py`, lisez `donnees/questions.csv` et affichez, avec `Counter`, le nombre de questions par intention attendue. Quelle intention est la moins représentée ?

Partie 14. Chercher les bonnes fiches et répondre avec

Un modèle ne connaît pas vos recettes. Si on lui demande le temps de préparation des crêpes du site, il invente. La solution s'appelle la **génération augmentée par recherche** (RAG, en anglais) : on cherche d'abord les fiches utiles dans vos données, on les colle dans le message système, et on demande au modèle de répondre **uniquement** à partir d'elles. Les vrais systèmes cherchent avec des vecteurs ; vous allez le faire avec des mots communs, ce qui suffit à comprendre le principe.

14.1 Chercher par mots communs

Créez `recherche.py`. La fonction `mots` réduit un texte à l'**ensemble** (`set`, une collection sans doublon et sans ordre) de ses mots utiles, sans accent ni majuscule, en retirant les mots trop courts ou trop fréquents. Première moitié :

recherche.py

```
# recherche.py : retrouver les recettes les plus proches d'une question
import re
import unicodedata
from recettes import charger

MOTS_VIDES = {"les", "des", "une", "est", "pour", "avec", "dans", "comment", "combien",
              "faire", "quoi", "que", "qui", "mon", "mes", "peux", "faut", "sans", "temps"}


def mots(texte):
    """Rend l'ensemble des mots utiles d'un texte, sans accents ni majuscules."""
    texte = unicodedata.normalize("NFD", texte.lower())
    texte = "".join(c for c in texte if not unicodedata.combining(c))
    return {m for m in re.findall(r"[a-z]+", texte) if len(m) >= 3 and m not in MOTS_VIDES}


```

- `unicodedata.normalize("NFD", ...)` sépare chaque lettre accentuée en lettre plus accent ; la ligne suivante jette les accents. « Crêpes » devient « crepes ».
- `re.findall(r"[a-z]+", texte)` découpe le texte en suites de lettres : la ponctuation disparaît.

Seconde moitié, à la suite :

recherche.py

```
    (lignes 1 à 16 inchangées)
def chercher(question, recettes, k=3):
    """Rend au plus k recettes, de la plus proche à la moins proche."""
    q = mots(question)
    notes = []
    for r in recettes:
        score = len(q & mots(r["titre"] + " " + " ".join(r["ingredients"])))
        if score > 0:
            notes.append((score, r))
    notes.sort(key=lambda paire: paire[0], reverse=True)
    return [r for score, r in notes[:k]]


def fiche(recette):
    """Met une recette au format texte que le modèle lira."""
    return (f"### {recette['titre']}\n"
            f"temps : {recette['temps_min']}\n"
            f"ingrédients : {', '.join(recette['ingredients'])}")


if __name__ == "__main__":
    toutes = charger()
    print(mots("Combien de temps pour faire des Crêpes ?"))
    for question in ["J'ai des champignons et du riz", "Combien de temps pour les crêpes ?",
                     "Vous livrez à Lyon ?"]:
        print(question, "=>", [r["titre"] for r in chercher(question, toutes)])
    print(fiche(toutes[12]))
```

- `q & mots(...)` est l'**intersection** de deux ensembles : les mots présents dans la question et dans la recette. Leur nombre sert de score.
- `notes.sort(key=lambda paire: paire[0], reverse=True)` trie du meilleur score au moins bon. Une `lambda` est une petite fonction sans nom, écrite sur place, ici « prendre le premier élément de la paire ».
- `fiche` met une recette au format texte que le modèle lira.

Windows (PowerShell), terminal 2	macOS (Terminal), terminal 2
`python recherche.py`	`python recherche.py`

Terminal 2 : vos programmes

```
(.venv) PS C:\Users\Public\Documents\M1\atelier-python> python recherche.py
{'crepes'}
J'ai des champignons et du riz => ['Risotto aux champignons', 'Omelette aux champignons', 'Chili sin carne']
Combien de temps pour les crêpes ? => ['Crêpes']
Vous livrez à Lyon ? => []
### Crêpes
temps : 30
ingrédients : farine, oeufs, lait, beurre, sucre
(.venv) PS C:\Users\Public\Documents\M1\atelier-python>
```

De toute la question sur les crêpes, il ne reste que `crepes` : les autres mots sont trop courts ou dans `MOTS_VIDES`. Riz et champignons trouvent le risotto en tête, qui contient les deux. Lyon ne trouve rien, et c'est la bonne réponse : aucune fiche ne parle de livraison.

14.2 L'assistant en ligne de commande

`assistant.py` assemble tout : il cherche les fiches, les place dans le message système sous un titre `CONTEXTE`, appelle le modèle, et rend la réponse avec les titres des fiches utilisées. S'il ne trouve aucune fiche, il n'appelle pas le modèle du tout : pas de fiche, pas de réponse inventée. La question se passe en **argument** de la commande, grâce au module `argparse`. Première moitié :

assistant.py

```
# assistant.py : répondre à une question en s'appuyant sur les fiches de recettes
import argparse
from openai import OpenAI
import config
from recettes import charger
from recherche import chercher, fiche

client = OpenAI(base_url=config.URL, api_key=config.CLE)


```

Seconde moitié, à la suite :

assistant.py

```
(lignes 1 à 10 inchangées)
def repondre(question, recettes):
    """Rend la réponse du modèle et les titres des fiches qu'on lui a données."""
    trouvees = chercher(question, recettes)
    if not trouvees:
        return "Aucune fiche ne correspond à votre question.", []
    contexte = "CONTEXTE\n" + "\n\n".join(fiche(r) for r in trouvees)
    reponse = client.chat.completions.create(
        model=config.NOM,
        temperature=0,
        messages=[
            {"role": "system", "content": "Réponds uniquement à partir des fiches suivantes.\n" + contexte},
            {"role": "user", "content": question},
        ],
    )
    return reponse.choices[0].message.content, [r["titre"] for r in trouvees]


if __name__ == "__main__":
    parseur = argparse.ArgumentParser(description="Assistant de recettes appuyé sur les fiches.")
    parseur.add_argument("question", help="la question, entre guillemets")
    parseur.add_argument("--sources", action="store_true", help="affiche les fiches utilisées")
    args = parseur.parse_args()
    texte, titres = repondre(args.question, charger())
    print(texte)
    if args.sources:
        print("Fiches utilisées :", ", ".join(titres) or "aucune")
```

- `repondre` rend deux valeurs à la fois, un texte et une liste : c'est un **tuple**, que l'appelant récupère avec `texte, titres = ...`.
- `add_argument("question")` déclare un argument obligatoire ; `"--sources"` avec `action="store_true"` déclare une option qui vaut `True` si elle est présente.

14.3 Utiliser l'assistant

`argparse` fabrique tout seul l'aide de la commande, et refuse un appel incomplet :

Windows (PowerShell), terminal 2	macOS (Terminal), terminal 2
`python assistant.py -h` `python assistant.py`	`python assistant.py -h` `python assistant.py`

Terminal 2 : vos programmes

```
(.venv) PS C:\Users\Public\Documents\M1\atelier-python> python assistant.py -h
usage: assistant.py [-h] [--sources] question

Assistant de recettes appuyé sur les fiches.

positional arguments:
  question    la question, entre guillemets

options:
  -h, --help  show this help message and exit
  --sources   affiche les fiches utilisées
(.venv) PS C:\Users\Public\Documents\M1\atelier-python>
```

Terminal 2 : vos programmes

```
(.venv) PS C:\Users\Public\Documents\M1\atelier-python> python assistant.py
usage: assistant.py [-h] [--sources] question
assistant.py: error: the following arguments are required: question
(.venv) PS C:\Users\Public\Documents\M1\atelier-python>
```

Posez maintenant de vraies questions, entre guillemets doubles pour que la phrase entière forme un seul argument :

Windows (PowerShell), terminal 2	macOS (Terminal), terminal 2

`python assistant.py "Combien de temps pour les crêpes ?" --sources`
`python assistant.py "J'ai des champignons et du riz, je fais quoi ?"`
`python assistant.py "Vous livrez à Lyon ?" --sources`
	les mêmes trois commandes

Terminal 2 : vos programmes

```
(.venv) PS C:\Users\Public\Documents\M1\atelier-python> python assistant.py "Combien de temps pour les crêpes ?" --so
urces
D'après la fiche « Crêpes », comptez 30 minutes. Il vous faut : farine, oeufs, lait, beurre, sucre.
Fiches utilisées : Crêpes
(.venv) PS C:\Users\Public\Documents\M1\atelier-python> python assistant.py "J'ai des champignons et du riz, je fais
quoi ?"
D'après la fiche « Risotto aux champignons », comptez 35 minutes. Il vous faut : riz arborio, champignons, bouillon,
parmesan, vin blanc.
(.venv) PS C:\Users\Public\Documents\M1\atelier-python> python assistant.py "Vous livrez à Lyon ?" --sources
Aucune fiche ne correspond à votre question.
Fiches utilisées : aucune
(.venv) PS C:\Users\Public\Documents\M1\atelier-python>
```

Les deux premières réponses citent leur fiche, la troisième refuse faute de fiche : **une réponse qu'on peut vérifier, ou pas de réponse**.

À vous (E15)

Posez à l'assistant, avec `--sources`, les questions « Une idée de dessert au chocolat ? » et « Que faire avec des lentilles ? ». Notez les réponses et les fiches utilisées. Puis trouvez une question sur une recette qui existe mais que la recherche ne retrouve pas, et expliquez pourquoi en vous servant de la fonction `mots`.

Partie 15. Faire le point et rendre le travail

15.1 Le dossier complet

Windows (PowerShell), terminal 2	macOS (Terminal), terminal 2
`Get-ChildItem -Recurse -Name | Where-Object { $_ -notmatch "venv|pycache|pytest_cache" }`	`find . -not -path "./.venv*" -not -path "*__pycache__*" -not -path "./.pytest_cache*" | sort`

Terminal 2 : vos programmes

```
(.venv) PS C:\Users\Public\Documents\M1\atelier-python> Get-ChildItem -Recurse -Name | Where-Object { $_ -notmatch "v
env|pycache|pytest_cache" }
donnees
tests
.env
.env.exemple
.gitignore
appel_requests.py
appel_sdk.py
assistant.py
bases.py
classer.py
config.py
erreurs.py
evaluer.py
explorer.py
modele_local.py
recettes.py
recherche.py
requirements.txt
resultats.csv
schemas.py
utiliser.py
verifier.py
donnees\questions.csv
donnees\recettes.json
tests\test_classer.py
tests\test_recettes.py
(.venv) PS C:\Users\Public\Documents\M1\atelier-python>
```

La commande liste tout le dossier en retirant les dossiers techniques. Comparez avec le vôtre : il doit y avoir les mêmes fichiers, plus `reponses.txt` et `compter.py`.

15.2 Arrêter proprement

Dans le terminal 1, Ctrl+C arrête le modèle. Dans le terminal 2, `deactivate` quitte l'environnement virtuel, sous Windows comme sous macOS :

Terminal 2 : vos programmes

```
(.venv) PS C:\Users\Public\Documents\M1\atelier-python> deactivate
PS C:\Users\Public\Documents\M1\atelier-python>
```

La prochaine fois, il suffira d'ouvrir le dossier dans VS Code et d'activer l'environnement : rien n'est à réinstaller.

15.3 Les erreurs fréquentes et leur remède

Ce que vous voyez	Ce que cela veut dire, et quoi faire
`python` n'est pas reconnu, ou Windows ouvre le Microsoft Store	Python n'est pas dans le PATH. Réinstallez en cochant « Add python.exe to PATH », puis fermez et rouvrez le terminal (partie 0).
`zsh: command not found: python` sur Mac	Hors environnement virtuel, la commande est `python3`. Activez l'environnement, ou tapez `python3`.
`Activate.ps1 ne peut pas être chargé, UnauthorizedAccess`	La règle d'exécution de PowerShell. Tapez une fois la commande `Set-ExecutionPolicy` de la section 2.3.
`ModuleNotFoundError: No module named 'openai'` (ou `pydantic`, `dotenv`)	L'environnement n'est pas actif dans ce terminal. Regardez l'invite : pas de `(.venv)`, activez-le. Réinstaller n'est presque jamais la solution.
`can't open file ... No such file or directory`	Le terminal n'est pas dans `atelier-python`. Vérifiez avec `Get-Location` ou `pwd`, revenez dans le dossier.
`IndentationError` ou `SyntaxError`	Espaces mal alignés, deux points oubliés à la fin d'un `if`, `for` ou `def`, guillemets typographiques venus d'un copier-coller. Regardez la ligne indiquée et celle du dessus.
`ModuleNotFoundError: No module named 'recettes'` dans pytest	Lancez `python -m pytest` et non `pytest`, depuis `atelier-python`.
« Le modèle ne répond pas », `ConnectionError`, `Connection refused`	Le modèle n'est pas lancé. Terminal 1 : `python modele_local.py`, et laissez-le ouvert.
« Le port 8000 est déjà pris »	Le modèle tourne déjà dans un autre terminal. Utilisez celui-là, ou arrêtez-le avec Ctrl+C.
Code 401, « Clé absente ou invalide »	Le fichier `.env` est absent, mal nommé (`.env.txt` sous Windows), ou la clé ne commence pas par `sk-`. Vérifiez avec `python config.py`.
Accents abîmés : « idÃ©e »	Un fichier UTF-8 lu avec un autre encodage. En PowerShell, ajoutez `-Encoding UTF8` ; en Python, `encoding="utf-8"` à chaque `open()`.
Imports soulignés en rouge dans VS Code, alors que le programme marche	L'éditeur utilise un autre interpréteur. Ctrl+Maj+P (Cmd+Maj+P), *Python: Select Interpreter*, choisissez celui de `.venv`.

Sous Windows, l'explorateur de fichiers masque les extensions : un fichier créé depuis l'explorateur sous le nom `.env` peut s'appeler en réalité `.env.txt`. Créez toujours ces fichiers depuis VS Code.

15.4 Ce que vous savez faire maintenant

Compétence	Où vous l'avez pratiquée
Installer Python, isoler un projet, figer ses dépendances	`.venv`, `pip`, `requirements.txt`
Manipuler les données d'une API : listes, dictionnaires, JSON	`bases.py`, `explorer.py`
Organiser son code en modules réutilisables	`recettes.py`, `utiliser.py`
Traiter les erreurs au lieu de les subir	`erreurs.py`, `appel_requests.py`
Protéger une clé d'API	`.env`, `.env.exemple`, `.gitignore`, `config.py`
Appeler un modèle en HTTP et avec le SDK, tenir une conversation	`appel_requests.py`, `appel_sdk.py`
Exiger une sortie structurée et la valider	`schemas.py`, `classer.py`
Tester du code qui dépend d'un modèle, sans le modèle	`tests/`
Mesurer la qualité d'un système d'IA	`evaluer.py`, `resultats.csv`
Faire répondre un modèle à partir de vos données	`recherche.py`, `assistant.py`

Pour passer à un vrai fournisseur, trois lignes changent, toutes dans `.env` : l'adresse, la clé, le nom du modèle. Aucun fichier Python ne bouge. C'est le bénéfice de la partie 8.

15.5 À rendre

Compressez le dossier `atelier-python` en une archive zip, **après avoir retiré** le dossier `.venv` et le fichier `.env`, et déposez-la à l'endroit indiqué par votre formateur. L'archive doit contenir :

- tous vos fichiers `.py`, y compris le dossier `tests` et `compter.py` ;
- `requirements.txt`, régénéré après votre dernière installation ;
- `.env.exemple` et `.gitignore` ;
- `resultats.csv` produit par votre dernière évaluation ;
- `reponses.txt`, avec vos réponses aux exercices E1 à E15, numérotées.

Avant de rendre, vérifiez deux choses : `python -m pytest -q` doit afficher uniquement des tests réussis, et aucune clé ne doit figurer dans vos fichiers `.py`.

Windows (PowerShell)	macOS (Terminal)
`Select-String -Path *.py -Pattern "sk-"`	`grep -n "sk-" *.py`

La commande ne doit rien afficher. Si elle trouve une ligne, la clé est écrite dans le code : déplacez-la dans `.env`.